

Ülesanne 2

Sisukord

1 Mis peab olema ülesande tulemusena tehtud?	1
1.1 Erinevad võimalused soovitud tulemuseni jõudmiseks	2
2 Esimesed sammud	4
2.1 Otsustuspuu, kuidas alustada	4
2.2 Ohud ja ebamugavused	5
2.3 Töö alustamise näide EAs	6
3 Millele andmebaasi füüsilise disaini mudeli täpsustamisel tähelepanu pöörata?	11
3.1 Kooskõla teiste mudelitega	11
3.2 Diagrammid	11
3.3 Klassifikaatorid	14
3.4 Skeemiobjektide, veergude ja kitsenduste identifikaatorid e nimed	14
3.5 Isikunimed andmebaasis hoidmine	18
3.6 Parooli andmebaasis hoidmine	20
3.7 Veergude järjekord	21
3.8 Andmetüübid ja väljapikkused	22
3.9 Piltide andmebaasis hoidmine	28
3.10 Veergude kohustuslikkus	29
3.11 Primaarvõtmed ja unikaalsuse kitsendused	31
3.12 Välisvõtmed	34
3.13 Hierarhiliste andmete andmebaasis hoidmine	35
3.14 Vaikimisi väärtused	37
4 Lisainfo	39

1 Mis peab olema ülesande tulemusena tehtud?

Ülesande eesmärgiks on saada korda oma iseseisva töö projekti andmebaasi füüsilist disaini kirjeldavad diagrammid e skeemid (välja arvatud CHECK kitsendused ja indeksid, mida pole vaja selle ülesande lahendamise käigus kirjeldada). Vajadusel tuleb kooskõla säilitamiseks täiendada kontseptuaalset andmemudelit. Andmebaasi diagrammid on näiteprojekti jaotises 3.1.

Ülesannete 2 ja 3 laiemaks eesmärgiks on saada CASE vahendis andmebaasi füüsilise disaini mudel sellisesse korda, et sellest koodi genereerimisel on saadud kood võimalik kas parandusteta või minimaalsete paranduste ja täiendustega käivitada ning tulemuseks on hästi disainitud baastabelitega andmebaas. Samuti peavad olema andmebaasi diagrammid nii ilmeka ja visuaalselt hästi esitatud, et need saab kopeerida portree-orientatsioonis A4 formaadis dokumenti ning tulemus on hästi loetav.

Sisuliselt tegelete Te *programmeerimise* ja *programmi silumisega*, ainult et tekstilise lähtekoodi asemel töötate selle visuaalse esitusega UMLis. Selleks, et programmeerida, on vaja *nõudeid*. Nõuded on antud juhul "Andmebaasid I" projektis või õppejõult saadud projektis (edaspidi lähteprojekt) – kontseptuaalses andmemudelis ja mujal. Nii selle kui järgnevate ülesannete täitmisel tuleb selle dokumendiga konsulteerida ja see peaks kogu aeg töö käigus kõrval lahti olema.

Väga paljud ülesandes ettenähtavad tegevused on mõeldud andmebaasi disainist või selle mudelitena esitusest *halvasti lõhnavate* kohtade eemaldamiseks ja selle kaudu *tehnilise võla* vähendamiseks. Väga paljudele nendele halbadele lõhnadele ja parandustele leidub vaste Robert C. Martini puhta koodi raamatus.

Martin, R. C., 2008. *Clean Code: A Handbook of Agile Software Craftsmanship*. 1st ed. New York – Prentice Hall.

Seda raamatut saate lugeda O'Reilly digitaalse platvormi kaudu e-raamatuna.

<https://learning.oreilly.com/library/view/clean-code/9780136083238/?ar>

Kuidas kasutada O'Reilly digitaalset platvormi?

https://maurus.ttu.ee/vaata_kysimust.php?kysimus=684

Tööd tuleb teha siin ja edaspidi oma projekti kaaslastega koos rühmatööna (kui käite samas praktikumis) või kui käite erinevates praktikumides, siis tuleb jätkata tööd, mida samal nädalal varem praktikumis käinud kaaslased alustasid.

Väljaspool praktikumi korrastage dokumenti. Eesmärk on, et Teie kirjatöö vormistus järgiks võimalikult täpselt lõputööde kirjutamise juhendit, mille leiate aadressil: <https://taltech.ee/lopetajale#p2529> (alajaotus +Lõputöö vormistamine)

Kontrollige ja vajadusel parandage oma dokumenti järgmiste reeglite alusel.

- ⤴ Kõik esimese taseme pealkirjad peavad algama uuel leheküljelt.
- ⤴ "Kõigis peatükkides ja alapeatükkides peab pealkirjale järgnema tekst. Uue alapealkirja, joonise või tabeli esitamine koheselt pärast pealkirja ei ole lubatud – pealkirjale järgneval real on alati tekst."
- ⤴ Kõikidel joonistel on allkirjad ning tabelitel on pealkirjad.
- ⤴ Kõikidele joonistele ja tabelitele viidatakse töö tekstis.
- ⤴ Kõikidele kasutatud materjalidele viidatakse töö tekstis.

Olen kohendanud projekti töövihikut ja näiteprojekti, et olla nende nõuetega kooskõlas.

1.1 Erinevad võimalused soovitud tulemuseni jõudmiseks

Kui kasutate CASE vahendit Enterprise Architect, siis on Teil tegelikult erinevad võimalused, kuidas saavutada ülesannete 2–4 oodatav tulemus.

1. Teha koopia lähteprojektis loodud andmebaasi tabelite disaini mudelist ja jätkata tööd koopiaga. Täiustatud mudelist tuleb genereerida andmekirjelduskeele lausete kood, mida tuleb siluda ja siis käivitada. **Sellist lahendust kirjeldavad ülesanded 2–4.**
2. Kui lähteprojektis loodud andmebaasi disaini mudel ei vastanud hästi tegelikult loodud andmebaasile, siis luua pöördprojekteerimist (*reverse engineering*) kasutades lähteprojektis realiseeritud andmebaasi põhjal andmebaasi disaini mudel. Jätkata tööd selle mudeliga vastavalt ülesannetes 2–4 esitatud juhendile.
3. Luua otse andmebaasi tabelid. Tabelite disainimisel peaks arvestama kõigi ülesannetes 2–4 väljatoodud põhimõtetega. Lõpuks tuleb

pöördprojekteerimist (*reverse engineering*) kasutades luua loodud tabelite põhjal andmebaasi disaini mudel ja esitada see diagrammidel vastavalt ülesandes 2 jaotises 3.2 toodud juhendile. Need diagrammid lähevad lähteprojekti peatükki 3.

- ⤴ Olemasoleva andmebaasi disaini visualiseerimine ilma CASE vahendit kasutamata:
 - https://maurus.ttu.ee/vaata_kysimust.php?kysimus=2845
 - Selleks, et luua iga registri kohta eraldi digramm(id), saab kasutada näiteks DBeaverit.
- ⤴ Pöördprojekteerimine EA vahendis:
 - https://sparxsystems.com/enterprise_architect_user_guide/12.0/database_engineering/importdatabaseschemafromod.html
- ⤴ Tabelite (ja ka muude andmebaasiobjektide) loomiseks on näiteks vähemalt ühes projektis kunagi kasutatud sellist andmebaasi evolutsioneerimise ning migreerimise tarkvara:
 - <https://maurus.ttu.ee/download.php?aine=346&document=37779&tyyp=do> (projekti autorite kogemused)
 - <https://knexjs.org/guide/migrations.html#migration-cli>
 - <https://alviandk.com/blog/simplifying-database-migration-with-knexjs-a-comprehensive-guide>
- ⤴ PostgreSQL andmebaasi kitsenduste nimede ühtlustamiseks saaks kasutada PostgreSQL andmebaasi laiendust:
https://github.com/katrinaibast/constr_name_unif, mida kirjeldatakse selles lõputöös: <https://digikogu.taltech.ee/et/Item/05157bb4-763e-4a9d-bc50-2025692b8e1d>

2 Esimesed sammud

Mida on vaja teha ja teada, enne kui andmebaasi tehnilise kavandamisega e disainimisega pihta hakata?

2.1 Otsustuspuu, kuidas alustada

- Soovin kasutada **Enterprise Architect (EA) CASE** vahendit.
 - **Lähteprojekt** on tehtud **EA** abil.
 - Soovin kasutada **Oracle** andmebaasisüsteemi.
 - Juhul kui lähteprojekt *ei ole* tehtud Oracles, siis teen selles genereeritud tabelite kirjeldust sisaldavast paketist/pakettidest koopia ning määran koopias kõikide tabelite puhul, et need realiseeritakse andmebaasisüsteemis Oracle.
 - Juhul kui lähteprojekt *on* tehtud Oracles, siis alustan selles genereeritud andmebaasi füüsilise disaini mudeli parandamist.
 - Soovin kasutada **PostgreSQL** andmebaasisüsteemi.
 - Juhul kui lähteprojekt *ei ole* tehtud PostgreSQLis, siis teen selles genereeritud tabelite kirjeldust sisaldavast paketist/pakettidest koopia ning määran koopias kõikide tabelite puhul, et need realiseeritakse andmebaasisüsteemis PostgreSQL.
 - Juhul kui lähteprojekt *on* tehtud PostgreSQLis, siis alustan lähteprojektis genereeritud andmebaasi füüsilise disaini mudeli parandamist.
 - **Lähteprojekt** on tehtud **RR** abil.
 - Loen "Andmebaasid I" kodulehe dokumendist <https://maurus.ttu.ee/download.php?aine=388&document=37555&tyyp=do> kuidas minna RR kasutamisele üle EA kasutamisele. Kui olen ülemineku lõpetanud, siis jätkan otsustusmudeli EA harust
 - Soovin kasutada **Rational Rose (RR) CASE** vahendit.
 - **Lähteprojekt** on tehtud **RR** abil.
 - Soovin kasutada **Oracle** andmebaasisüsteemi.
 - Juhul kui lähteprojekt *ei ole* tehtud Oracles, siis genereerin kontseptuaalsest andmemudelist uue andmebaasi füüsilise disaini mudeli, määrates andmebaasisüsteemiks Oracle. Viited videotele genereerimise kohta on "Andmebaasid I" kodulehe kataloogis https://maurus.ttu.ee/vaata_doc_voog.php?m_doc_catalog=4844&aine=388
 - Juhul kui lähteprojekt *on* tehtud Oracles, siis alustan lähteprojektis genereeritud andmebaasi füüsilise disaini mudeli parandamist.
 - Soovin kasutada **PostgreSQL** andmebaasisüsteemi.
 - Juhul kui lähteprojektis *on* määratud andmebaasi füüsilise disaini mudelis andmebaasisüsteemiks ANSI SQL 92, siis alustan lähteprojektis genereeritud andmebaasi füüsilise disaini mudeli parandamist.
 - Juhul kui lähteprojektis *ei ole* määratud andmebaasi füüsilise disaini mudelis andmebaasisüsteemiks ANSI SQL 92, siis genereerin

kontseptuaalsest andmemudelist uue andmebaasi füüsilise disaini mudeli, määrates andmebaasisüsteemiks ANSI SQL 92. Videod genereerimise kohta on "Andmebaasid I" kodulehe kataloogis https://maurus.ttu.ee/vaata_doc_voog.php?m_doc_catalog=4844&aine=388

Kui Te teete projekti **mõne muu CASE vahendiga kui EA või RR**, siis tuleb valida selline CASE vahend, milles tehtud andmebaasi diagrammides saate esitada samasugust informatsiooni kui EA või RR vahendis tehtud diagrammides.

Kui Teil on **olemas PostgreSQL või Oracle andmebaas**, kuid **puudub andmebaasi füüsilise disaini mudel**, siis võite selle esimese versiooni genereerimiseks EA vahendis kasutada pöördprojekteerimist (*reverse engineering*).

https://sparxsystems.com/enterprise_architect_user_guide/12.1/database_engineering/importdatabaseschemafromod.html

2.2 Ohud ja ebamugavused

Kahjuks on nii RR kui EA CASE vahendis loodud andmebaasi disaini mudelitega seoses õhus *riknemise* oht.

RR korral avaldub riknemine selliselt, et tehes tabeli kirjeldusel topeltklõpsu, ei avane tabeli kirjelduse aken, kui soovite andmebaasi kirjeldusest genereerida SQL laused, siis see ei õnnestu. Te ei saa disaini kirjeldust enam muuta. Mudel rikneb, sest mõni selle element rikneb. Kui suudate (*katse-eksituse meetodil*) selle elemendi üles leida ja ära kustutada, siis saate mudeliga normaalselt edasi töötada. EA korral kaovad ära näiteks välisvõtme kitsendused. Uuesti loomise, salvestamise ning faili uuesti avamise järel on need jälle kadunud. Ei jäägi muud üle, kui disaini mudel uuesti luua.

Ühel üliõpilaste rühmal RR korral probleem (vähemalt ajutiselt) kadus, kui mudelis eemaldati skeemi (*schema*) nimest ##. Seega, palun ärge igaks juhuks kasutage RR mudelis skeemi nimedes # märke. Näiteks skeemi nime C##TUD2 asemel kasutage nime TUD2.

NB! Sellega seoses järgmine soovitus – enne kui hakkate mudelifailiga tööle, tehke sellest koopia. Kui fail peaks riknema, siis saate koopiat kasutades faili suhteliselt värske versiooni taastada. Soovitan sellise koopia teha iga praktikumi alguses ja ka iga kord kui väljapool koolitunde selle failiga tööle hakkate. Kasutage koopia failide nimedes koopia tegemise kuupäeva (nt **Diagrammid_2025_10_02.eap**) ja säilitage koopiaid kuni semestri lõpuni.

Hoiatus! EAs on välisvõtmete muutmine väga ebamugav ning välisvõtme kitsenduste kirjeldused võivad väga kergesti rikneda.

Riknenud seoste parandamise õpetus EA jaoks on siin:

https://maurus.ttu.ee/vaata_kysimust.php?kysimus=2772

RR pole (võrreldes EAg) üldse paha ja mõni asi on seal ka parem/mugavam kui EAs. Seega nii RR kui EA sobivad antud aines kasutamiseks.

2.3 Töö alustamise näide EAs

Töö alustamist selgitavad videod, kus tehakse läbi selles jaotises kirjeldatavad tegevused.

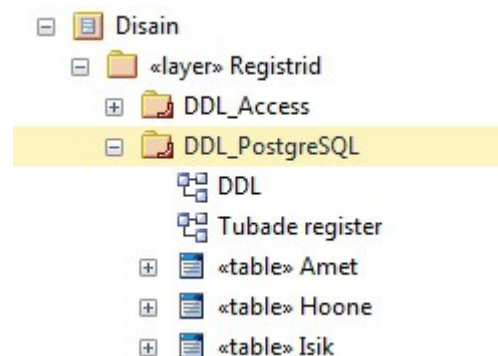
- EA 12:
 - [EA \(ver 12\) vahendis füüsilise disaini andmemudeli migreerimisega alustamine.mp4](#)
- EA 16:
 - [EA \(ver 16\) vahendis füüsilise disaini andmemudeli migreerimisega alustamine.mp4](#)

Mõelge UML paketist kui failisüsteemi kataloogist. Te loote katalooge, et:

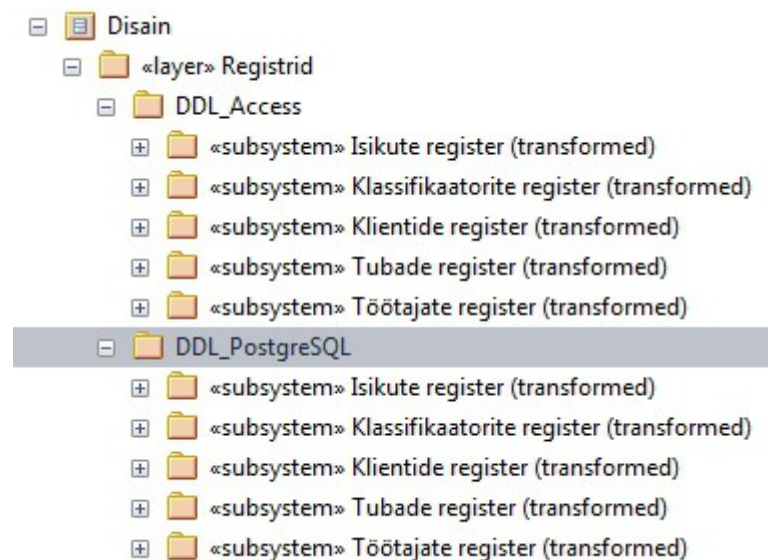
- faile organiseerida ja lihtsamini üles leida,
- vältida failide nimekonflikte.

Järgnevalt illustreeritakse võimalikku andmebaasi füüsilise disaini mudeli pakettide struktuuri EAs. Paketi nimede *DDL_Access* ja *DDL_PostgreSQL* asemel võib ka kasutada mõnda muud nime (nt *MS Access* ja *PostgreSQL*) – oluline on, et nime järgi saaks lihtsalt aru, millise andmebaasisüsteemi jaoks mõeldud tabelite kirjeldust see pakett sisaldab.

Juhul, kui kasutasite tabelite kirjelduse saamiseks EA vaikimisi mudeliteisendust.



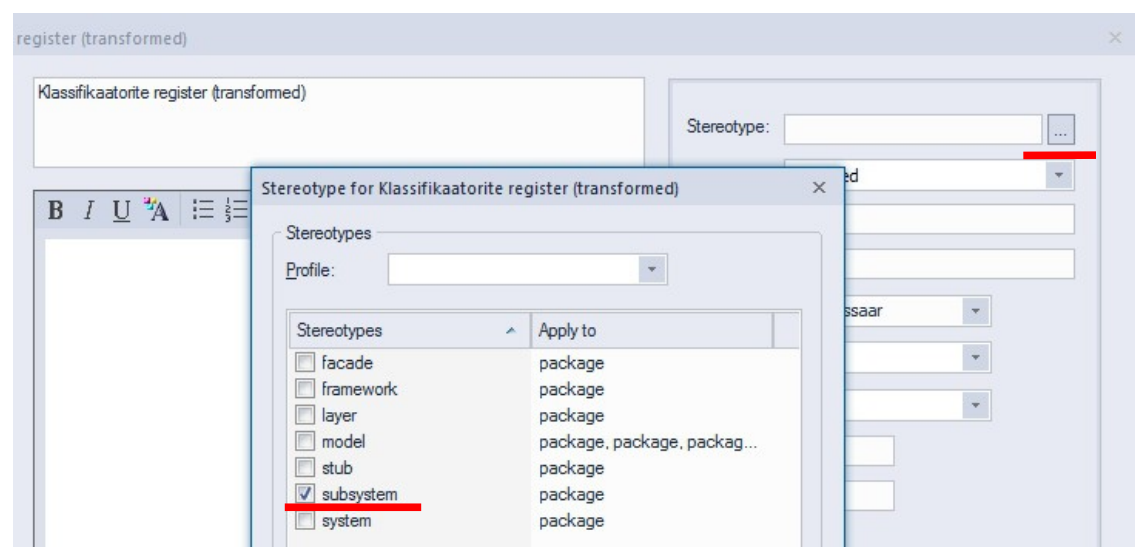
Juhul, kui kasutasite tabelite kirjelduse saamiseks "Andmebaasid I" aines ja ka selles dokumendis eelnevalt kirjeldatud täiustatud mudeliteisendust.



Pange tähele, et registreid (andmekesksed allüsteemid) tähistavatele pakettidele on stereotüübiks määratud <<subsystem>>. See on tehtud ilmekuse huvides, et mudeli vaatelejal oleks kohe selge, mida see pakett näitab. Samal põhjusel oln paketil *Registrid*, stereotüüp <<layer>>, sest registrite näol on tegemist äriarhitektuuri ühe kihiga.

Kuidas sellise struktuurini jõuda? Järgnevad ekraanitõmmised illustreerivad EAs tabelite kirjeldust sisaldavast pakettist koopia tegemist, pakettide ümbernimetamist ning tabeliga seotud andmebaasisüsteemi määramist.

Paketi stereotüübi määramine.

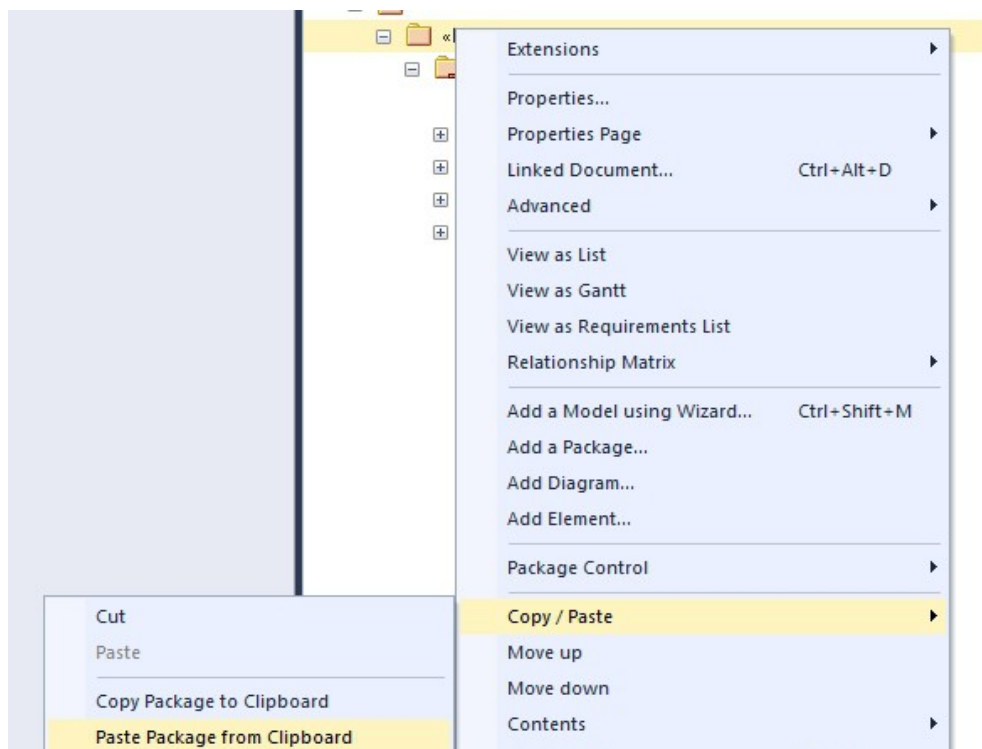
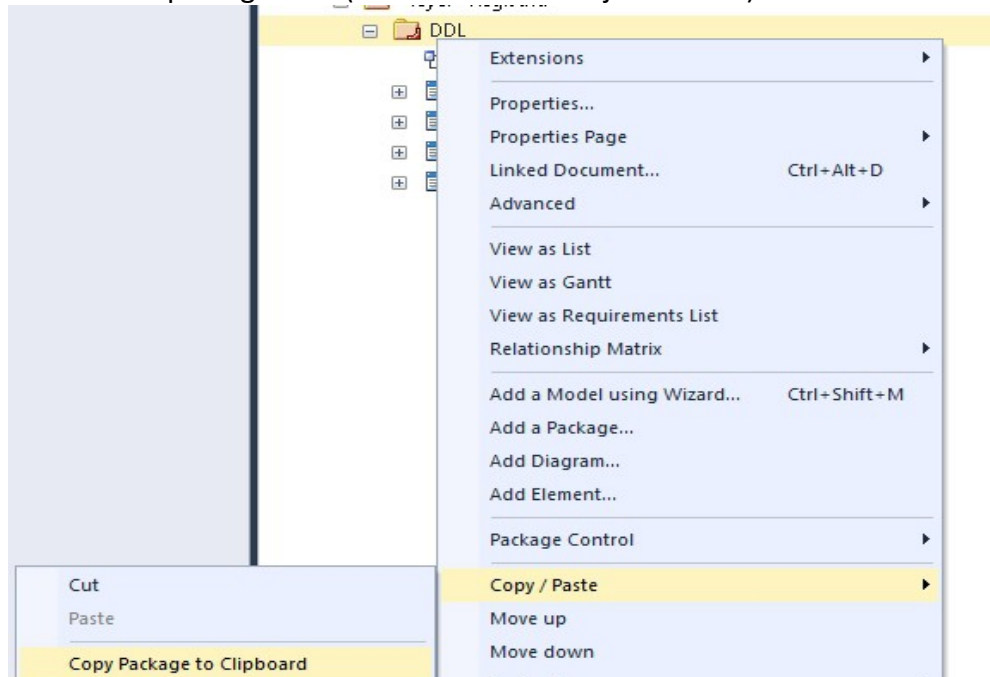


EA uuemates versioonides tuleb stereotüübi *subsystem* leidmiseks valida avanenud aknas:

Perspective: All Perspectives

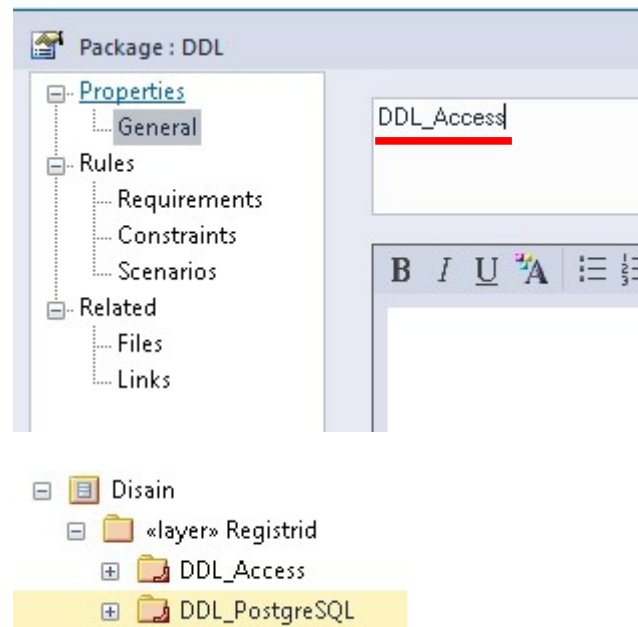
Profile: <<none>>

Paketist koopia tegemine (töötab ka CTRL + C ja CTRL + V).



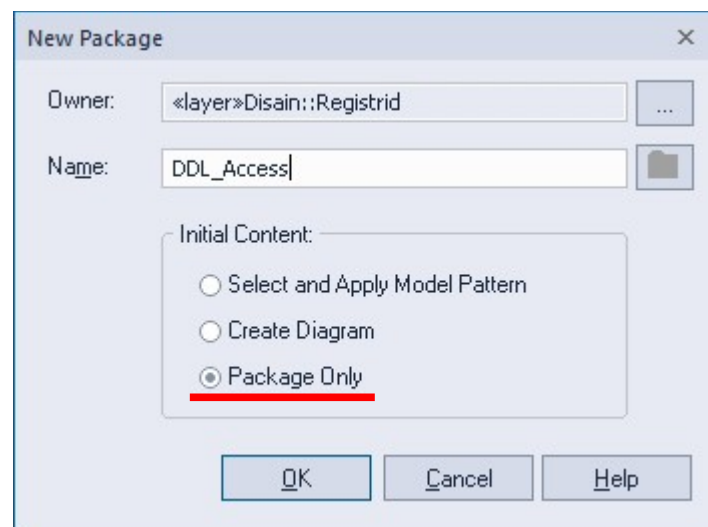
Pakettide ümbernimetamine, et oleks selge, millises pakettis on millise andmebaasisüsteemi jaoks mõeldud andmebaasi füüsilise disaini kirjeldus.



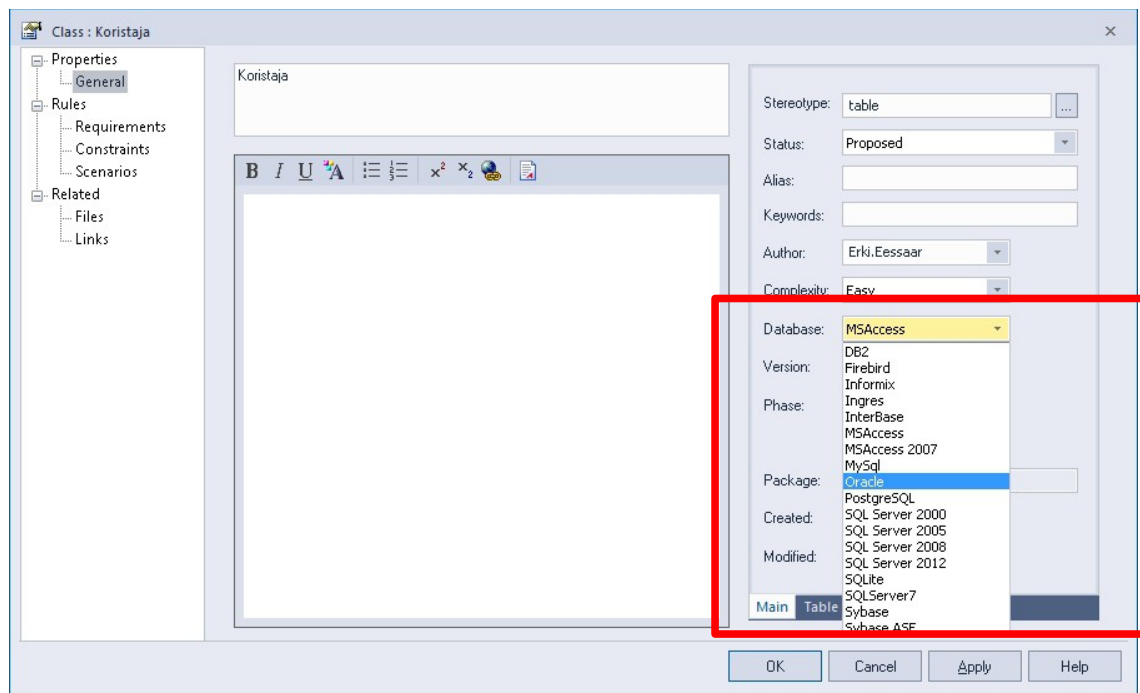


Veenduge alati, et avate vaatamiseks õiges pakettis olevad diagrammid.

Kui on vaja lisada uus tühi pakett:



Tabeliga seotud andmebaasisüsteemi muutmine. Selle tulemusena muudab CASE vahend eeldefineeritud vastavustabeli alusel veergude tüüpe. Kahjuks on tulemus ebakvaliteetne ja seal on vaja teha palju muudatusi. Andmebaasisüsteemi muudatus tuleb teha kõikides kopeeritud pakettide tabelite kirjeldustes. Muudatuse tulemusena saab tabeli veergude kirjeldamisel valida andmetüübi tabeli kirjelduses määratud uue andmebaasisüsteemi andmetüüpide hulgast.



Küsimus: Mida teha, kui tabeli kirjelduse avamisel uuemas EA versioonis (nt EA 15) ei pakuta andmebaasisüsteemi muutmise valikut?

Vastus: Põhjus tundub olevat selles, et tabelit esitava klassi puhul on stereotüübi <<table>> ja klassi vaheline seos katki. Sellest tulenevalt ei saa CASE vahend aru, et see klass esitab tabelit ja peab seda endiselt "tavaliseks" klassiks. Sellest tulenevalt saab modelleerija valida programmeerimiskeelt, mitte andmebaasisüsteemi. Parandamiseks tehke järgnevat (iga tabeli korral eraldi).

- Määrake sellele klassile mistahes muu stereotüüp (valik akna ülemisest paremast servast). Kinnitage valik, vajutades stereotüübi valimise aknas OK.
- Määrake klassile tagasi stereotüüp <<table>>. Nüüd peaks CASE vahend pakkuma andmebaasisüsteemi valimise võimalust.

Vaadake hääletut lühivideot EA 15 põhjal:

- [010 Tabeliga seotud andmebaasisüsteemi muutmine EA15.mp4](#)

3 Millele andmebaasi füüsilise disaini mudeli täpsustamisel tähelepanu pöörata?

Lugege järgnev läbi. Väga võimalik, et osad asjad (nt diagrammideks jaotus, sobiva struktuuriga klassifikaatori tabelid) on juba lähteprojektis tehtud ja siis pole vaja midagi muuta.

3.1 Kooskõla teiste mudelitega

🚩 Kontseptuaalne andmemudel ja andmebaasi füüsilise disaini mudel peavad olema kooskõlas. Kõik, mis on disainitaseme andmemudelis (v.a disainitaseme abitabelid), peab olema tuletatav teisendusreeglite rakendamise tulemusena kontseptuaalsest andmemudelist. Kui tahate muuta tabelite struktuuri, et näiteks alustada mingite uute andmete registreerimist, siis tuleb see muudatus viia sisse ka kontseptuaalsesse andmemudellisse ning kui nende andmete registreerimine toimub projekti teemaks olevas funktsionaalses allsüsteemis, siis ka teistesse mudelitesse.

3.2 Diagrammid

PostgreSQL või Oracle andmebaasi disaini kirjeldavad diagrammid e skeemid tuleb Teil lõpuks panna projekti dokumendi kolmandasse peatükki – lähteprojekti andmebaasi disaini diagrammide asemele.

🚩 Eesmärk: Kõik diagrammid peavad olema dokumendis (A4 formaadis portree-orientatsiooniga leht) 100% suurendusega loetavad. Järgnevalt kirjeldatakse, kuidas seda andmebaasi füüsilise disaini diagrammide korral saavutada.

- **Kui Te pole seda veel teinud**, siis looge iga funktsionaalse allsüsteemi poolt kasutatava või teenindatava registri kohta eraldi andmebaasi diagrammid (vt näiteprojekti). Teiste sõnadega, looge vähemalt üks diagramm iga alajaotuses "**1.2.2 Seosed pädevusalade ja registritega**" esitatud registri kohta.

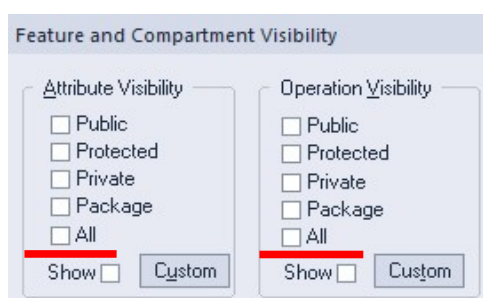
- Iga registri kohta tuleb luua üks või mitu diagrammi (mitu siis, kui ühe diagrammi jaoks on elemente liiga palju või on neid ühelt diagrammilt halb lugeda). Näiteks ühel diagrammil on kõik klassifikaatorite tabelid (kui äriarhitektuuris on *klassifikaatorite register*). Teisel diagrammil on tabelid töötajate kohta (kui äriarhitektuuris on *töötajate register*) jne. Kui klassifikaatorite tabelleid on ühel diagrammil esitamiseks liiga palju, siis tehke mitu diagrammi – näiteks ühel diagrammil on ainult seisundiklassifikaatorid.
- Igal registri R kohta loodud diagrammil on registrisse R kuuluvad tabelid + nende tabelitega **otseselt** välisvõtmete kaudu seotud tabelid, **kui välisvõti on mõnes registrisse R kuuluvas tabelis**.

🚩 Registrisse R kuuluvate tabelite korral on näha veerud, kitsendused ja indeksid.

✚ Registre R *mittekuuluvate* tabelite korral on veerud, kitsendused ja indeksid **peidetud**.

- Tabeli veergude ja kitsenduste peitmiseks RR andmebaasi diagrammil aktiveerige diagrammil olev tabeli kirjeldus ning valige parema hiireklahvi alt avanevast menüüst:
 - *Options => Suppress Columns*
 - *Options => Suppress Triggers*
- Tabeli veergude ja kitsenduste peitmiseks EA andmebaasi diagrammil aktiveerige diagrammil olev tabeli kirjeldus ning valige parema hiireklahvi alt avanevast menüüst:
 - *Features & Properties => Feature and Compartment Visibility...*

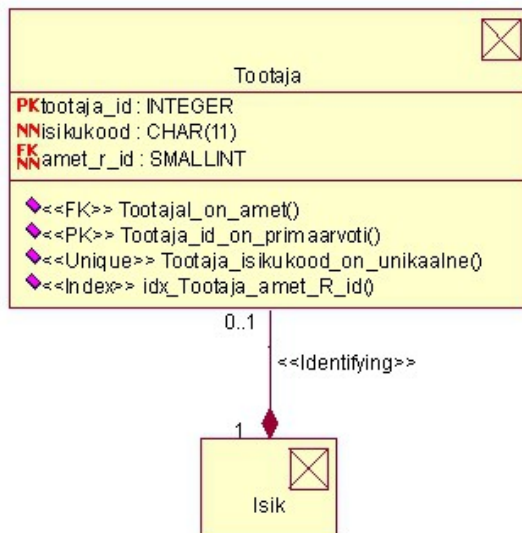
✚ Avanenud akna vasakus ülemises nurgas eemaldage märgendid valikute "All" eest:



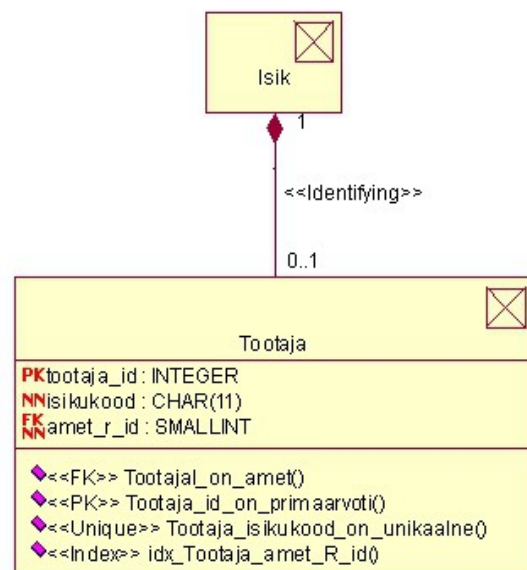
- Kui toimite eelnevalt toodud juhiste järgi, siis on täidetud ka järgmised üldised nõuded.
 - Rusikareegli kohaselt ei peaks diagrammil olema üle 5+/-2 elemendi (antud juhul tabeli). Pidage meeles eesmärki, milleks on hästi loetav diagramm. See rusikareegel on lihtsalt abivahend selle eesmärgini jõudmiseks. Vabalt võib olla, et viis suure veergude/kitsenduste arvuga tabelit on liiga palju ja kümme väikese veergude/kitsenduste arvuga tabelit on täiesti piisav.
 - Diagrammid peavad olema osalise ülekattega, st neil on mõned ühised elemendid. See aitab erinevad diagrammid mõttes tervikuks kokku siduda.
 - Kui esitate ühe ja sama tabeli kirjelduse mitmel diagrammil, siis tuleks selle tabeli veerge, kitsendusi ja indekseid näidata vaid ühel diagrammil ([Ainult Üks Kord](#) disainipõhimõtte).
 - Iga tabeli detailid (veerud, kitsendused, indeksid) on näha täpselt ühel diagrammil. Tabeli detailideta esitus koos seostega võib olla nähtav rohkem kui ühel diagrammil.

✚ Paigutage loetavuse huvides diagrammil tabelid nii, et ülatüübile vastav tabel ei oleks madalamal kui selle alamtüüpidele vastavad tabelid. Seega, kui *Isik* on ülatüüp ning *Klient* ja *Töötaja* on selle alamtüübid, siis on tabel *Isik* diagrammil kõrgemal (või vähemalt samal tasemel) kui tabelid *Tootaja* ja *Klient*.

Halvem paigutus diagrammil

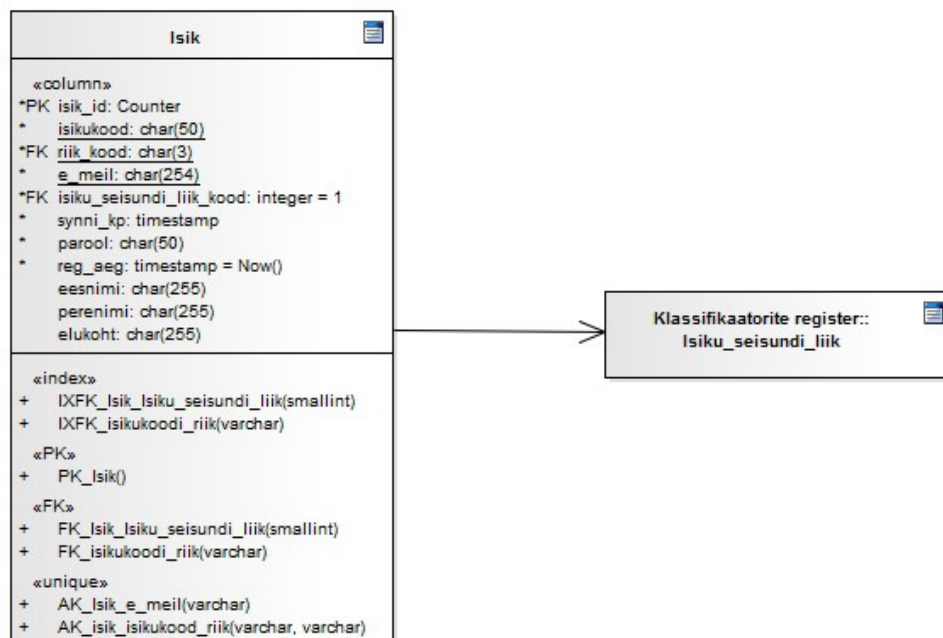


Parem paigutus diagrammil

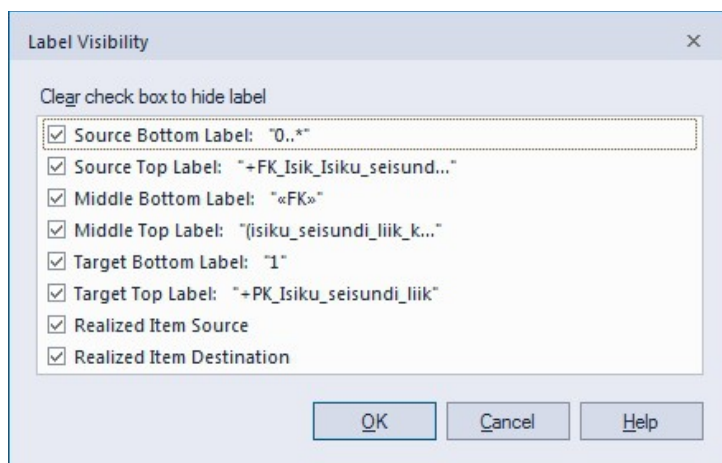


🚩 Nii nagu Te annate sisukad nimed registritele, tabelitele või veergudele, tuleb sisukad nimed anda ka diagrammidele.

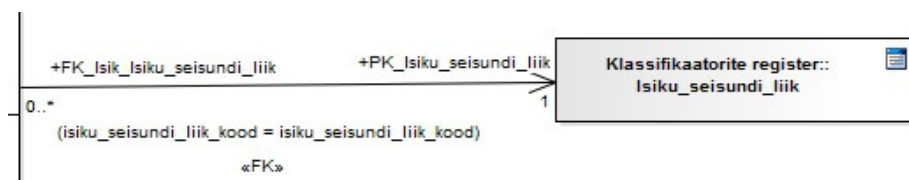
🚩 Tagage EA vahendis, et välisvõtmete kohta väljastataks diagrammil võimalikult palju informatsiooni. Kui tabelite vaheline seos näeb diagrammil välja nii



, siis EA12 korral klõpsake seosel ning valige parempoolse hiireklahvi alt hüpikmenüüst *Visibility => Set Label Visibility* Märgistage kõik valikud:



Veenduge, et oleks nähtav valik *Visibility => Hide All Labels* (st *Show All Labels* on sisse lülitatud). Seose visuaalne esitus muutub diagrammil selliseks:



3.3 Klassifikaatorid

Üldine klassifikaatorite tabel ei sobi. Andmebaasi füüsilise disaini mudel tuleb genereerida nii, et seda ei tekiks. Kui see tabel on andmebaasi füüsilise disaini mudelis varasemast ajast olemas, siis tuleb see tabel kustutada ning muuta andmebaasi füüsilise disaini mudelit nii, et igale klassifikaatori tüübile vastab eraldi tabel, kus on kõik kontseptuaalsest andmemudelist tulenevad veerud.

Huvi (pole kohustuslik!) korral lugege selle kohta, kuidas esitada klassifikaatoreid SQL-andmebaasides lugege järgnevast bakalaureusetööst.

Vellemaa, A.S., 2015. *Mõned disainimustrid klassifikaatorite esitamiseks SQL andmebaasides*. Bakalaureusetöö. TTÜ Informaatikainstituut. [WWW] <https://digi.lib.ttu.ee/i/?3484> (29.08.2025)

3.4 Skeemiobjektide, veergude ja kitsenduste identifikaatorid e nimed

Valikuline: Kui soovite, et andmebaasi disaini mudelis (ja selle põhjal realiseeritud andmebaasis) kasutatakse andmebaasiobjektide nimetamisel teist keelt kui on kasutusel analüüsi mudelites (näiteks analüüsi mudelid e nõuded on eesti keeles, kuid andmebaasiobjekte tahate nimetada inglise keeles), siis tuleb andmebaasi disaini mudelis kõik nimed ära muuta.

- Keelevahetus tuleb teha käsitsi. Vähemalt meie kasutatavad CASE vahendid seda ei automatiseeri.
- Keelevahetus peab olema täielik – sama taseme tulemites (nt disaini mudelid, kood), tuleb läbivalt kasutada ühte keelt. See, kui *näiteks* osade tabelite nimed

on eesti keeles, osade nimed on inglise keeles või tabelite nimed on inglise keeles, kuid indeksite nimed on eesti keeles on väga halb stiil.

- Vahetades keelt disaini mudelis ei pea analüüsi mudelis midagi muutma.
 - Analüüsi mudelid esitavad nõudeid. On täiesti loomulik, et need pannakse kirja nõuete allikale (kliendile) kõige arusaadavamas keeles ning kasutades tema mõistestikku. See, et kliendi esindajad vaatavad koostatud nõuete kirjeldust üle või löövad isegi selle koostamises kaasa, on loomulik protsess.
- ✚ Kui annate andmebaasiobjektidele eestikeelsed nimed, siis hindamisel selle eest miinuseid **EI SAA** (inglise keele eest **EI SAA** lisapunkte).
- ✚ Ükskõik mis keelt kasutate – hea nimetamise põhimõtted on ühesugused.

✚ Lugege seda dokumenti, järgige selle soovitusi ja vältige selles kirjeldatud vigu: <https://bonesmoses.org/2016/pg-phriday-all-in-a-name/> Kuigi see artikkel on kirjutatud PostgreSQL jaoks, kehtib see ka Oracle ning kõigi teiste SQL-andmebaasisüsteemide korral.

✚ Nimed peavad lähtuma mingist (Teie valitud) reeglistikust, mida tuleb kõigi nimede puhul **järjekindlalt** järgida. Soovitan need reeglid kuhugi kirja panna ja projekti liikmete vahel laiali jagada, et kõik neid järgiksid.

✚ Nimed peavad vastama valitud andmebaasisüsteemi reeglitele (seega näiteks kõik tähikud tuleb eemaldada – piiritletud identifikaatorid, kus need on lubatud, on ebasoovitavad).

PostgreSQL: <https://www.postgresql.org/docs/current/sql-syntax-lexical.html#SQL-SYNTAX-IDENTIFIERS>

Oracle: https://docs.oracle.com/database/121/SQLRF/sql_elements008.htm

✚ Ärge kasutage läbisegi erinevaid kirjastiile.

- [S/s]nake_case
- camelCase
- PascalCase

Soovitan kasutada **[S/s]nake_case**, sest andmebaasiobjektide regulaarsed identifikaatorid salvestatakse andmebaasi süsteemikataloogis kas suur- või väiketähtedega. Genereerides andmebaasi põhjal tagasi andmebaasikeele lauseid, on tulemuseks (sõltuvalt andmebaasisüsteemist) identifikaatorid nagu: CAMELCASE, camelcase, PASCALCASE, pascalcase, SNAKE_CASE, snake_case.

✚ Kuigi see pole andmebaasisüsteemi nõue, siis soovitan asendada ka täpitähed (õõääöüü). Sõltuvalt kasutatavast tehnoloogiast pole välistatud probleemid programmides, mis peavad kasutama selliste nimedega andmebaasiobjekte. Samuti võib tekkida probleem andmebaasisüsteemi vahetamisel, kui üks süsteem lubab identifikaatorites täpitähti ja teine mitte. Jällegi – asendustes tuleb olla järjekindel, teha seda igal pool ning kasutades igal pool ühesugust asendust.

✚ Suur- ja väiketähtedest. Juhul kui andmebaasiobjekti loomise lauses pole nimi jutumärkides (st nime näol on tegemist regulaarse identifikaatoriga), siis andmebaasis on nimi tõstutundetu (*case insensitive*). See tähendab, et

andmebaasi kasutamise mõttes pole vahet, kas nimes kasutati suurtähti, väiketähti või nende kombinatsiooni. Lause `CREATE TABLE Isik ...` loob tabeli, mille põhjal saab teha päringut nii `SELECT * FROM Isik`; nii `SELECT * FROM ISIK`; kui ka nii `SELECT * FROM iSiK`;. Andmebaasi kasutatavuse seisukohalt tuleks eelistada tõstutundetuid nimesid tõstutundlikele (st ülesandes 4 on vaja hoolitseda, et SQL koodis ei oleks nimed jutumärkides – jutumärkides nimi on tõstutundlik). Mudelite ja koodi loetavuse seisukohalt oleks siiski hea, kui olete nimeses suur ja väiketähtede kasutamisel järjekindel. Järgnevalt esitan mõned soovitusel (pole reeglid; peaaegu – olge järjekindel):

- Ärge kasutage nimes läbivalt suurtähti (ISIK). Sotsiaalmeedias seostatakse seda karjumisega: https://en.wikipedia.org/wiki/All_caps ning need võivad ka jätta vale mulje, et tegemist on akronüümiga nagu EL või NATO või CASE (Vikipeedia kohaselt on akronüüm "sõnana häälduv ühend, mis on moodustunud mitme sõna algustähtedest"). **NB!** Kui muudate EA12 Oracle andmebaasi disaini mudelis objekti (nt tabel, veerg) nime, siis muudab CASE vahend automaatselt seal kõik tähed suurtähtedeks. Hea oleks, kui siis oleks tabelite disaini mudelis KÕIK nimed suurtähtedega.
- Tabeli nimes võiks olla esitähed suurtähed ja ülejäänud tähed väiketähed (nt *Isiku_seisundi_liik*).
- Veergude nimes võiks kasutada läbivalt väiketähti (nt *isiku_seisundi_liik_kood*).
- Kui kitsenduse nimes on tabeli(te) nimed, siis ka see (need) võiksid alata suurtähega ja veergude nimed vastavalt väiketähega (nt *chk_Isik_eesnimi_perenimi*).
- Tüübi tähise puhul võiks läbivalt kasutada ühesugust suur- ja väiketähtede kombinatsiooni (nt *PK_Isiku_seisundi_liik* – *PK* kõikides nimeses suurtähtedega).

🚩 SQL-andmebaasis esitatakse andmed tabelites, kus on read ja veerud. Mõttetu on kasutada tabelite (sh tuletatud tabelite) nimeses üldmõistetele viitavaid termineid nagu "tabel", "andmed", "info" ning veergude nimeses üldmõistetele viitavaid termineid nagu "veerg", "andmed", "info".

- **Halvem:**
 - Ruumi_andmed
- **Parem:**
 - Ruum

🚩 Ärge kasutage liiga üldiseid veergude nimesid.

- **Halb:**
 - id
 - kood
 - tüüp
 - aeg
 - kp
- **Parem:**
 - isik_id
 - riik_kood
 - kaup_tüüp

- reg_aeg
- synni_kp

🚩 Lihtsustamiseks tabelite ühendamise lausete kirjutamist:

- andke välisvõtme veergudele ja nende poolt viidatavatele võtmeveergudele ühesugused nimed. Selle soovitusel vastu *peab* eksima, kui välisvõti viitab sama tabeli kandidaatvõtmele (rekursiivne seos), sest nimega tabelis ei tohi olla mitu samanimelist veergu. Selle soovitusel vastu *võib* eksida, kui välisvõtme veeru nimi kirjeldab primaartabeli *rolli* välisvõtmele vastava seosetüübi kontekstis – näiteks tabelis *Riik* on primaarvõtme veerg *riik_kood* ja tabelis *Isik* on välisvõtme veerg *isikukoodi_riik*.

- **Halvem:**

🚩 Välisvõtme veeru nimi: riik

🚩 Välisvõtme poolt viidatava veeru nimi: riik_kood

- **Parem:**

🚩 Välisvõtme veeru nimi: riik_kood

🚩 Välisvõtme poolt viidatava veeru nimi: riik_kood

- **Kontranäide**

🚩 **Halvem:**

- Tabelis *Treening* välisvõtme veeru nimi *isik_id*, mis on seotud tabeli *Tootaja* veeruga *isik_id*.

- Treening ja isik võivad olla seotud väga erinevatel põhjustel – isik võib näiteks olla treeningu registreerija, viimane andmete muutja, treeningu kinnitaja, treener, treeningu eest vastutaja jne. Veeru nimest ei tule see põhjus välja ja see halvendab inimkasutaja jaoks tabeli struktuurist arusaamist.

🚩 **Parem:**

- Tabelis *Treening* välisvõtme veeru nimi *registreerija_id*

- Ärge kasutage erinevate tabelite võtme ja välisvõtme veergude nimedes segamini erinevaid käandeid.

🚩 **Halb:**

- isiku_id
- kaup_id

- klient_kood
- kauba_kood

🚩 **Parem:**

- isik_id
- kaup_id

- kliendi_kood
- kauba_kood

🚩 Kitsendustele tuleb anda sisukad nimed. Kui CASE vahend lisab kitsenduste nimedesse numbreid, siis tuleb need eemaldada.

🚩 Kitsenduste nimed peaksid olema skeemi piires unikaalsed. Arvestage võimalusega, et erinevates tabelites võib olla sama nimega veerge – seega kitsenduse nime unikaalsuse tagamiseks skeemi piires ei piisa veeru nime

kasutamisest kitsenduse nimes. Kitsenduse nime unikaalsuse tagamiseks lisage kitsenduse nimesse tabeli nimi, millega see kitsendus on seotud. Miks on kitsenduste nimede unikaalsus vajalik kirjeldatakse pikemalt siin: https://maurus.ttu.ee/vaata_kysimust.php?kysimus=2921

⚠️ Oracle puhul jälgige, et nimed (sh kitsenduste nimed) poleks pikemad kui **30 baiti** (märki). Kontrollimiseks võite näiteks kasutada:

<https://mothereff.in/byte-counter> (PostgreSQLis on *apex2.taltech.ee* serveris kasutusel vaikimisi maksimaalne pikkus – **63 baiti**; Oracle lubab *enamike* nimede puhul (teatud tüüpi andmebaasiobjektide korral peab nimi olema siiski lühem) maksimaalse pikkusena 128 baiti alates versioonist 12.2, kuid serveris on versioon 12.1.0.1).

⚠️ Selleks, et näha maksimaalset PostgreSQL nime pikkust serveris, käivitage andmebaasis:

◦ SHOW max_identifier_length;

⚠️ Kui PostgreSQLis käivitatavas SQL lauses (nt CREATE TABLE, SELECT) on identifikaator lubatust pikem, siis lühendab PostgreSQL ise nime. Siiski on vajalik see nimi ise lühemaks teha – nii andmebaasis kui seda kirjeldavates mudelites.

⚠️ Lühendamise tulemusena ei pruugi nimi enam olla unikaalne.

⚠️ Lühendamine ei toimu psqli käskudes.

⚠️ Liiga pikka nime on tülikas lugeda, meelde jätta ja kirjutada – tõmmake paralleele:

⚠️ <https://www.historyrundown.com/top-5-people-with-the-longest-names/>

⚠️ Kuigi PostgreSQLis saaks maksimaalset nime pikkust muuta, muutes installeerimisel juhtparameetri NAMEDATALEN väärtust, siis ilmselt enamikel serveritel seda ei tehta. Seega andmebaasi teisele serverile kolides võib andmebaasisüsteem hakata liiga pikki nimesid lühendama.

◦ <https://pgpedia.info/n/NAMEDATALEN.html>

⚠️ Jälgige, et veeru nimi ja selles registreeritavate andmete sisu oleks kooskõlas.

◦ **Halb:**

⚠️ registreerimise_kp TIMESTAMP

◦ **Parem:**

⚠️ registreerimise_kp DATE

⚠️ Temporaalsete (ajaandmete) veergudele soovitan järgmist nimeskeemi.

⚠️ Kui veerus registreeritakse kuupäev + kellaaeg, siis veeru nimel järelliide *aeg* (nt alguse_aeg, registreerimise_aeg) (ingl k *time*).

⚠️ Kui veerus registreeritakse ainult kuupäev, siis veeru nimel järelliide *kp* (nt alguse_kp, registreerimise_kp) (ingl k *date*).

⚠️ Ärge kasutage kontseptuaalses andmemudelisis atribuutide nimedena ja tabelites veergude nimedena nimesid *aeg* ja *kp*, sest need on liiga üldised ja ei anna täpsemat infot vastavate andmete tähenduse kohta.

3.5 Isikunimede andmebaasis hoidmine

⚠️ Isikunimede registreerimisel tuleb arvestada võimalusega (nii kontseptuaalses andmemudelisis kui andmebaasi füüsilise disaini mudelisis), et kultuurilistest eripäradest tulenevalt pole isikul näiteks perenime, nimed on ootamatult pikad või

sisaldavad ootamatuid sõnu/sümboleid. Nimi *Null* ei tohiks rakendusi rivist välja lüüa.

- <https://www.bbc.com/future/article/20160325-the-names-that-break-computer-systems>
- <https://www.wired.com/2015/11/null/>
- <https://www.ohetuleht.ee/989305/hakka-voi-karpima-liiga-pikk-nimi-voib-smart-id-loomisel-probleeme-tekitada>
- https://maurus.ttu.ee/vaata_kysimust.php?kysimus=2930

Isikut tõendavas dokumendis olev nimi ja inimese tegelik nimi ei pruugi kattuda. Näiteks Eesti isikut tõendavate dokumentide seaduse § 91 lõige 2 ütleb, et "Kui isiku eesnimi ületab 15 tähemärki või perekonnanimi ületab 28 tähemärki, kantakse see dokumenti nii, et nime lõpust jäetakse kirjutamata tähed, mis andmevälja reale ei mahu." Lõige 3 ütleb, et lisaks kantakse nimi "algupärasel kujul dokumendi (välja arvatud isikutunnistus, digitaalne isikutunnistus ja elamisloakaart) märkustelehele ning kanne kinnitatakse pitsatiga." <https://www.riigiteataja.ee/akt/131122019016?leiaKehtiv> Kontseptuaalses andmemudel (nõuete mudel) tuleks täpsustada, kas isiku nime registreerimisel peetakse silmas tegelikku nime või dokumendis olevat nime.

Soovitan väga lugeda isikunimede ja nende SQL-andmebaasides hoidmise kohta käivat bakalaureusetööd.

Jõgi, M., 2016. *Mõned disainimustrid isikunimede hoidmiseks SQL-andmebaasides..* Bakalaureusetöö. TTÜ Informaatikainstituut. [WWW] <https://digi.lib.ttu.ee/i/?7072> (29.08.2025)

Kui registreerite eesnime ja perenime, siis peaks olema kasutatud ühte järgnevatest lahendustest. Veenduge, et kontseptuaalses andmemudel (nõuete mudel) olev atribuutide kirjeldus on tehtud valikuga kooskõlas (nii atribuutide kirjelduse tabel, kui ka olemi-suhte diagrammil olevad atribuutide võimsustikud).

- ⚡ Nii veerg *eesnimi* kui *perenimi* on mittekohustuslikud. Lisaks on jõustatud andmebaasis kitsendus, et vähemalt üks nendest peab olema registreeritud. See on parem lahendus (vt järgnevat ETISE näidet).
- ⚡ Veerg *eesnimi* on kohustuslik ja *perenimi* mittekohustuslik. Üks Jõgi järeldus: "Eesnimi on ainuke nimetüüp, mille olemasolu võiks eeldada erinevates nimekultuurides". Kuid vaadake järgnevat ETISE näidet – ma ei tea, kas puuduv eesnimi on sisestusviga või kultuuriline eripära.

Isikute otsing Eesti Teaduse Infosüsteemis (ETIS). <https://www.etis.ee/Portal/Persons/Index> Esimeses veerus on perenimi ja teises veerus on eesnimi. Isikud, kellel puudub perenimi.

-	Apneet Kaur	EST / ENG
-	Balpreet Kaur	EST / ENG
-	Mohammed Hasan	EST / ENG
-	Punyakoti Ganeshalah Veena	
-	Shweta Suran	EST / ENG
-	Suresh Kumar	EST / ENG

Isikud, kellel puudub eesnimi.

Farhat-ul-ain

EST / ENG

Eesti Vabariigi nimeseadusega (<https://www.riigiteataja.ee/akt/106032015032?leiaKehtiv>) "sätestatakse füüsilise isiku nime (edaspidi isikunime) andmise ja kohaldamise põhimõtted ja kord ning Eesti kodaniku ja Eestis viibiva välismaalase (edaspidi isiku) isikunime ühtse kasutamise alused." Seadus ütleb, et "Isikunimi koosneb eesnimest ja perekonnanimest." On tõlgendamise küsimus, kas see tähendab, et eesnimi ja perekonnanimi (perenimi) on mõlemad kohustuslikud või nimetab see kõigest isikunime võimalikke komponente. Seaduse kohaselt: "(3) Välisriigis antud või kohaldatud isikunime kasutusele võtmisel järgitakse käesoleva seaduse kohaseid isikunime andmisel isikunime suhtes kehtivaid nõudeid, kusjuures: 1) isiku nõusolekul säilitatakse talle välisriigis antud või kohaldatud isikunimi, kui isikunimi on kirjutatud eesti-ladina tähtedega; (4) Välisriigi dokumendis olev nimi, mis ei ole eesnime ega perekonnanime osa, loetakse eesnime osaks ja lisatakse eesnime järele." Järeldan, et isikud kellel on näiteks vaid üks nimi (https://en.wikipedia.org/wiki/Mononymous_person), võivad säilitada oma nime ning eesnime välja on üldjuhul vaja midagi kirjutada.

3.6 Parooli andmebaasis hoidmine

Andmebaasi tuleb kohandada tulenevalt vajadusest tuvastada süsteemi kasutajad ning lubada süsteemi kasutada ainult selleks volitatud kasutajatel ja ainult neile antud volituste piires. Järgnev lähtub eeldusest, et kasutate levinud (kuid paljude süsteemide jaoks juba liiga vähe turvalist) lahendust, mille korral süsteemi kasutajad registreeritakse andmebaasis ning lõppkasutajate ja andmebaasis loodud kasutajate vahel ei ole üks-ühele vastavust. Igal kasutajal peab olema unikaalne kasutajanimi. *kasutajanimi* võib olla eraldi atribuut/veerg. Samas, kui Te registreerite isiku meiliaadressi ja see on kohustuslik ning unikaalne (peaks nii olema), siis võite kasutada meiliaadressi kasutajanimena ning eraldi veergu *kasutajanimi* pole vaja.

Andmebaasis olev parool ei tohi olla avatekst, sest see on väga ebaturvaline. Avatekstilise parooli asemel tuleb andmebaasis hoida parooli alusel arvutatud räsiväärtust (*hash value*). Räsiväärtus on "püsipikkusega sõnumilühend, mis on andmetest saadud räsifunktsiooni abil" (<https://akit.cyber.ee/term/508>) Veerus *parool* lubatud märkide arv sõltub kasutatavast räsifunktsioonist. Kui Te ei kasuta lahendust, mis esitab soola parooli räsiväärtuse osana, siis peab leiduma ka soola hoidmiseks mõeldud veerg, milles olevad väärtused on nt 32 märgi pikkused juhuslikult valitud märkidest koosnevad stringid. Miks see vajalik on, lugege palun siit:

<https://www.martinstoeckli.ch/hash/en/>

Vt ka <https://www.php.net/manual/en/function.password-hash.php>

Kui soovite kasutada õppejõu väljapakutavaid tehnilisi lahendusi (sellega tegeleb ülesanne 10), mille korral arvutatakse parooli räsi andmebaasisüsteemi poolt, siis looge järgmiste omadustega veerud. Need veerud peaksid olema samas tabelis (nt *Kasutajakonto*).

PostgreSQL:

parool: VARCHAR(60) NOT NULL

Veergu *sool* ei ole vaja, sest *sool* integreeritakse algoritmi poolt parooli räsisse.

Oracle:

sool: VARCHAR2(32) NOT NULL

parool: VARCHAR2(128) NOT NULL

Vastavad atribuudid peavad olema ka kontseptuaalses andmemudelis. Kui neid seal ei ole (nt atribuuti *sool*), siis täiendage kontseptuaalses andmemudelis olemi suhte diagrammi ja atribuutide definitsiooni.

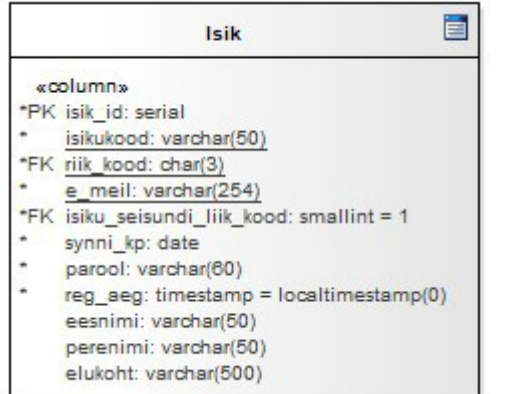
3.7 Veergude järjekord

- 🚩 Muutke tabelites veergude järjekorda nii,
 - et kõigepealt tuleb *primaarvõtme veerg/veerud*,
 - siis *alternatiivvõtmete veerud*,
 - siis esimesse kahte kategooriasse mitte kuuluvad *välisvõtmete veerud*,
 - siis ülejäänud *kohustuslikud veerud*
 - ning kõige lõpuks ülejäänud *mittekohustuslikud veerud*.
- 🚩 Sama kategooria loogiliselt kokkukuuluvad veerud (nt *eesnimi* ja *perenimi*; *kasutajanimi* või *e_meil*, *parool* ja *sool*; *laiuskraad* ja *pikkuskraad*) võiks paigutada kõrvuti, isegi kui see läheb vastuollu eelnevalt nimetatud üldreeglitega.
 - Kuna tavaks on esitada koordinaadid *laiuskraad/pikkuskraad* järjekorras, siis võiks veerg *laiuskraad* olla enne veergu *pikkuskraad*.

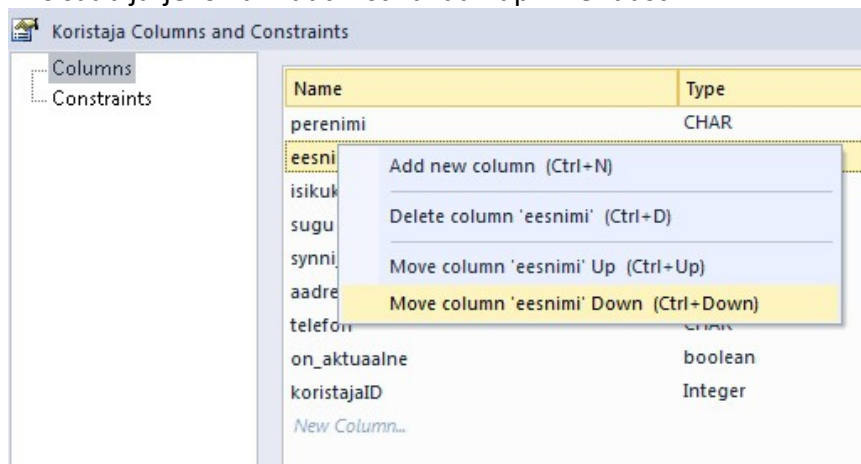
Veergude järjekord *ei mõjuta* andmete tähendust! Samas selline järjekord kiirendab mudeli ülevaatamist ning võimaldab lihtsamalt avastada puuduvaid veerge/kitsendusi. Samuti lihtsustab see andmebaasis tabelist ülevaate saamist ning selle kaudu ka päringute ja andmemuudatuse lausete kirjutamist. Veergude järjekorra kohta leidub erinevaid soovitusi: <https://stackoverflow.com/questions/2113553/ordering-columns-in-database-tables> Võite kasutada minu pakutust erinevat järjekorda, või siis midag muud – oluline on olla järjekindel. Siiski, üldise tavana peaksid primaarvõtme veerud olema tabelis esimesed.

Veergude järjekorraga seoses tuleb arvestada, et tabeli struktuur ei ole kivisse raiutud ning tabeli eluea jooksul võib tekkida vajadus lisada tabelisse uusi veerge. Selleks on ALTER TABLE lause. Uued veerud lisatakse tabeli lõppu. Head ja kõigis süsteemides ühtemoodi toimivat veergude järjekorra muutmise lahendust ei ole. ALTER TABLE lausega seda teha ei saa. See on kahetsusväärne, sest SQL omistab veergude järjekorrale tähtsust. Üks variant on muuta veergude järjekorda tabelile loodud vaadetes, kuid jätta tabelites see järjekord muutmata. Kui soovida ikkagi muuta järjekorda ka tabelites, siis siin on soovitusi:

- PostgreSQL jaoks: https://wiki.postgresql.org/wiki/Alter_column_position
- Oracle 12c jaoks: <https://stackoverflow.com/questions/4939735/re-order-columns-of-table-in-oracle> (põhineb peidetud veergude funktsionaalsusel, mis lisandus Oracle 12c)

Halvem veergude järjekord	Parem veergude järjekord
 <pre> «column» *FK riik_kood: char(3) * synni_kp: date * e_meil: varchar(254) *FK isiku_seisundi_liik_kood: smallint = 1 *PK isik_id: serial perenimi: varchar(50) elukoht: varchar(500) isikukood: varchar(50) reg_aeg: timestamp = localtime(0) eesnimi: varchar(50) </pre>	 <pre> «column» *PK isik_id: serial * isikukood: varchar(50) *FK riik_kood: char(3) * e_meil: varchar(254) *FK isiku_seisundi_liik_kood: smallint = 1 * synni_kp: date * parool: varchar(60) * reg_aeg: timestamp = localtime(0) eesnimi: varchar(50) perenimi: varchar(50) elukoht: varchar(500) </pre>

EAs saab järjekorra muutmist valida hüpikmenüüst.



3.8 Andmetüübid ja väljapikkused

- Iga veeru jaoks tuleb valida kõige sobivam andmetüüp andmebaasi realiseerimiseks kasutatava andmebaasisüsteemi poolt pakutavate tüüpide hulgast. Kui CASE vahend ei võimalda kõige sobivamat tüüpi valida, siis tuleb mees pidada, et muudatus tuleb teha genereeritud SQL koodis (pole selle ülesande teema).

PostgreSQL: <https://www.postgresql.org/docs/current/datatype.html>

Oracle: https://docs.oracle.com/database/121/SQLRF/sql_elements001.htm

RR vahendis saab sobiva tüübi valikus puudumisel selle sisestada otse diagrammile:

	Amet_R
PK	amet_r_id : SMALLINT
NN	nimetus : VARCHAR(70)
	<<>> kirjeldus : TEXT

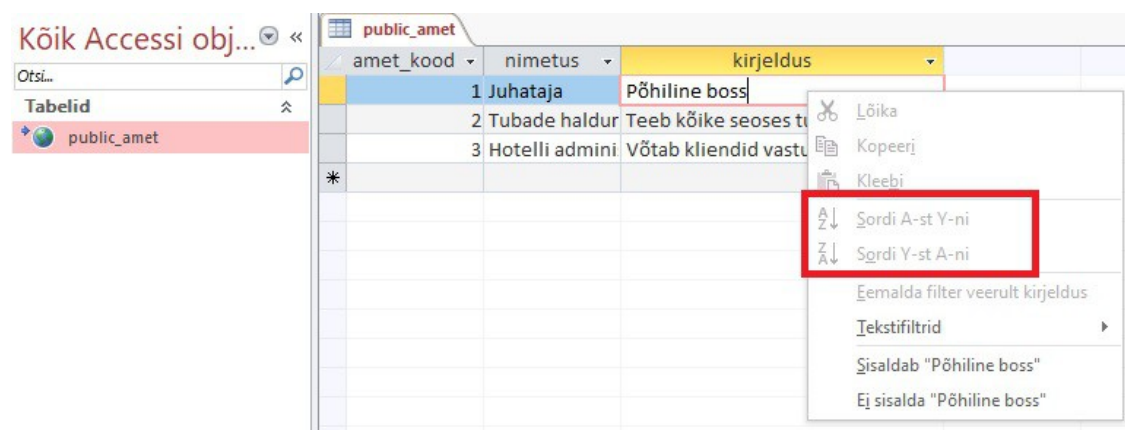
Taustainfo! Kui muudate EA tabelite disaini mudelis tabeliga seoses andmebaasisüsteemi, siis teeb EA esialgse andmetüüpide asenduse, kuid see asendus on **ebapiisav** ja **ebakvaliteetne**. Asendus tehakse EAs kirjeldatud andmetüüpide vastavuse alusel. Kuna see on ebapiisav, siis on ka tulemus mitterahuldav. EA12 tuleb kasutatud vastavuse *vaatamiseks* valida: *Project => Settings => Database Datatypes => Datatype Map*

Product	Datatype	Size Unit	Default	Max
MSAccess	Byte			
MSAccess	Counter			
MSAccess	Currency	Precision and Scale	(15,4)	15
MSAccess	DateTime			
MSAccess	Double			
MSAccess	Integer			
MSAccess	Long			
MSAccess	Long Integer			
MSAccess	Memo			
MSAccess	OLEObject			
MSAccess	Single			
MSAccess	Text	Length	50	255
MSAccess	YesNo			

🚩 Kuidas jõustada PostgreSQLis tekstilise veeru maksimaalset väljapikkust?

- <https://stackoverflow.com/questions/4848964/difference-between-text-and-varchar-character-varying>
- Määrata veeru tüübiks VARCHAR(n), kus n on maksimaalne märkide arv. n võib vajadusel olla väga suur, arvestades, et VARCHAR tüüpi väärtuse suurus võib olla kuni 1GB.
 - *Eelis:* Lihtne teha.
 - *Puudus:* Maksimaalse väljapikkuse muutmiseks tuleb kustutada veerust sõltuvad vaated, muuta väljapikkust ning luua vaated uuesti.
- Määrata veeru tüübiks TEXT ja lisada veerule maksimaalset pikkust piirav CHECK kitsendus.

- *Eelis*: Maksimaalse väljapikkuse muutmiseks piisab CHECK kitsenduse kustutamisest ja uue piiranguga uuesti loomisest.
 - *Puudus*: Andmebaasi kasutavad välised programmid (nt rakenduste generaatorid) ei oska selliselt määratud maksimaalset väljapikkust arvestada või seavad TEXT veeru kasutamisele piiranguid.
- 🔴 Näiteks kui lingite MS Accessi andmebaasi ODBC andmeühendust kasutades tabeli, kus on TEXT tüüpi veerg (järgnevas näites *kirjeldus*) ja VARCHAR(n) tüüpi veerg, siis hüpikmenüüst valitav MS Accessi sisseehitatud ridade sorteerimise võimekus toimib VARCHAR(n) veeru puhul, kuid ei toimi TEXT veeru puhul.



Kui genereerida MS Accessis lingitud tabeli põhjal vorm, siis on TEXT tüüpi veerul sisestusväli visuaalselt suurem kui VARCHAR(n) tüüpi veeru korral. Seda isegi siis, kui TEXT tüüpi veerul on maksimaalset väljapikkust piirav kitsendus.

amet_kood	1
nimetus	Juhataja
kirjeldus	Põhiline boss

- 🔴 Andmetüüpide ja väljapikkuste valimisel lähtuge põhimõttest – **nii suur kui vajalik (et kõik andmed saaks registreeritud) ja nii väike kui võimalik (et andmebaasiskeemi uurijal ei tekiks andmetest eksiarvamusi; et mitte lubada väärtuseid mis ilmselgelt on ekslikud; et mitte raisata salvestusruumi).** VARCHAR(n), VARCHAR2(n), CHAR(n) tüüpide korral tuleb valida sobiv maksimaalne väljapikkus. NUMBER(p), NUMBER(p,s), DECIMAL(p,s) tüüpide korral tuleb valida täpsus (*precision*, p) ja skaala (*scale*, s).
- Lähtemudelil võib olla kommentaari või kirjelduse veeru tüübiks VARCHAR(255) – 255 on maksimaalne tekstivälja suurus MS Accessis. PostgreSQLis ja Oracles on maksimaalne väljapikkus suurem ja selle valimisel tuleb arvestada reaalse maailma eripäradega. Mõelge sellele, et 255 märki on vähem kui kaks korda suurem X-i (endine Twitter) säutsu algsest maksimumpikkusest (140 märki) ning umbes sama pikk kui X-i säutsus on praegu lubatud

(<https://www.theverge.com/2017/11/7/16616076/twitter-280-characters-global-rollout>). Kas tõesti saab 255 märgiga kõik vajaliku kirja panna?

- Vabateksti väljade puhul, kus tekstipikkused pole ennustatavad, oleks parim valik tüüp TEXT (PostgreSQL) või CLOB (Oracle). Kui on soov/tahtmine lubatud teksti pikkust piirata, siis saab veerule defineerida CHECK kitsenduse, et väärtus selles veerus ei tohi olla pikem kui n märki. Kui seda pikkust on vaja muuta, siis see on palju lihtsam kui VARCHAR(n) puhul n muutmine. Meenub näiteks, kuidas <https://maurus.ttu.ee/software/> süsteemis osutus ühe kirjelduse välja pikkus peagi liiga väikeseks (kuigi palusin andmebaasi loomisel selle enda arvates **varuga** määrata). Kõigepealt jäi tänu sellele mõnda aega oluline info üliõpilastele edastamata ja lõpuks nägin kurja vaeva selle pikkuse suurendamisega.
- Kui kommentaaride ja kirjelduste puhul on VARCHAR(255) liiga väike, siis klassifikaatorite nimetuste puhul võib seda olla *liiga palju*. Mõelge sellele, et klassifikaatorite nimetusi hakkavad lõppkasutajad nägema andmete sisestamise vormides (liitboksides, hüpikakendes vms kohtades). Järgnevas näites näeb kasutaja voodi tüübi nimetuse esimest **129**-t märki. Kui klassifikaator ei ole riiklik või rahvusvaheline (nagu nt riikide koodid), siis on süsteemi arendajal vabadus klassifikaatorite nimetuste pikkuste üle ise otsustada.

Voodi tüüp

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean commodo ligula eget dolor. Aenean massa. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus.

- n, p, s väärtuste valimisel arvestage piirjuhtudega (maksimaalne võimalik väärtus, mida soovite registreerida). Vajadusel googeldage (näiteks otsingustring "what is the longest email address").
- n, p, s tuleb määrata nii suured kui vajalik ja nii väikesed kui võimalik. Läbimõeldud väärtused annavad skeemi uurijale (inimkasutaja või programm) taustainfot selles veerus oodatavate väärtuste kohta ning ühtlasi vähendavad võimalust registreerida ebakorrektsed andmeid. Samas on töötavas andmebaasis n, p ja s muutmine mõnikord tehniliselt tülikas ja suure andmete hulga korral aeganõudev ning tabelite teiste kasutajate poolt kasutamist takistav tegevus.
- Kuna tüübi CHAR korral lisatakse väärtuse lõppu tühikuid, et saavutada maksimaalne lubatud väljapikkus, siis seda tüüpi tuleks kasutada vaid juhul, kui on teada, et väärtused on alati ühesuguse pikkusega (nt riikide koodid).
- Andmebaas peaks võimaldama registreerida ka pikki isikunimesid (vaadake palun üle väljapikkused), st tulema toime piirjuhtudega.
 - Liiga lühikesed maksimaalselt lubatud väärtused võivad põhjustada kannatusi, segadust, lisatööd ja mainekadu (süsteemi omanikule):
 - <https://reisile.postimees.ee/4484884/reisija-pidi-lisatasu-maksma-sest-tema-nimi-ei-mahtunud-lennupiletile>
 - <https://www.bbc.com/news/world-us-canada-24097775>

- Üliharva esinevate ülipikkade isikunimede korral võib võtta seisukoha, et registreeritakse nime lühivorm (<https://www.historyrundown.com/top-5-people-with-the-longest-names/>). Liiga suur väljapikkus (nt eesnimi VARCHAR(1500)), mida praktikas ei lähe ilmselt kunagi vaja, tekitab asjatuid turvariske (keegi registreerib nimena ründeskripti) ja probleeme kasutajaliideses.
- Aadressi välja väljapikkuse valimisel arvestage, et ka kohanimed võivad olla pikad ning et aadress koosneb enamasti mitmest komponendist (nt linn, tänav, korter; maakond; vald; küla; talu), millest igaühel on oma nimi (https://en.wikipedia.org/wiki/List_of_long_place_names).
- Rahasummade korral on levinud soovitus kasutada tüüpi deklaratsiooni DECIMAL(19,4) (PostgreSQL; *decimal* ja *numeric* on sünonüümid ja viitavad samale tüübile) või NUMBER(19,4) (Oracle). Rahasummade puhul arvestage, et kaupade ja teenuste hinnad tuleb esitada *vähemalt* kahe komakoha täpsusega. Pidage silmas, et erinevatel valuutadel on ettenähtud erinev vaikimisi komakohtade arv (<https://web.archive.org/web/20230424221114/http://www.thefinancials.com/Default.aspx?SubSectionID=curformat> – euro korral vaikimisi kaks kohta peale koma). Näiteks väikesed tariifid, ühikuhinnad kaupadel, mida saab osta ainult suurtes kogustes ja mõned finantstehingud nõuavad rohkem kui kaks (kolm-neli) kohta peale koma. Kui hinnad on väga väikesed ja müüdavad kogused väga suured, siis lubage ümardamise probleemide vältimiseks kuni viis kohta peale koma. PostgreSQL pakub tüüpi *money*, kuid sellesse on sisse integreeritud valuuta tähis (sõltub serveri seadetest – *apex2.taltech.ee* serveris €, kuid võib ka olla nt \$) ning enne aritmeetikatehteid on vaja väärtus teisendada NUMERIC tüüpi. Seega soovitatakse *money* mitte kasutada ja tarvitada selle asemel püsikomatüüpi. Lugege lisaks:
 - <https://sakala.postimees.ee/386430/euroraha-harjutab-ostjaid-tuhandikega>
 - <https://stackoverflow.com/questions/27325620/decimal19-4-or-decimal19-2-which-should-i-use>
 - <https://www.riigiteataja.ee/akt/111022016017>
 - <https://arileht.delfi.ee/artikkel/80146622/4-99-voi-5-00-kuidas-napilt-madalam-hind-meie-ostuotsuseid-mojutab>
 Kuidas hinna esimene number ja komakohad mõjutavad ostuotsuseid?
 - <https://commandprompt.com/blog/postgresql-and-financial-calculations-part-two/>
 Kõige väiksema skaalaga arv arvutuses seab piirid tulemuse täpsusele. Seega peaksid ühes arvutuses olevad püsikomaarvud olema ühesuguse skaalaga. Mida suurem skaala, seda täpsemad on arvutused.
- Jälgige rahasummade ja teiste arvude korral, mille puhul arvutustes ja ümardamistes ei tohi olla vigu, et need *ei oleks* salvestatud ujukomatüüpi veerus, mille korral väärtused salvestatakse kahendsüsteemis
 - Peatükk 10 raamatus:
 -  https://www.ester.ee/record=b2888667*est
 -  <https://learning.oreilly.com/library/view/sql-antipatterns-volume/9798888650011/?ar>

- Kuidas kasutada O'Reilly digitaalset platvormi?
https://maurus.ttu.ee/vaata_kysimust.php?kysimus=684
 - <https://floating-point-gui.de/basic/>
 - Katsetage väärtuse kahendsüsteemis esitamist:
<https://www.h-schmidt.net/FloatConverter/IEEE754.html>
 - ▲ Näiteks väärtus **0.3** salvestatakse tegelikult väärtusena **0.300000011920928955078125**
 - <https://commandprompt.com/blog/postgresql-and-financial-calculations-part-one/>
 - ▲ PostgreSQLil põhinev näide, kuidas ujukomatüüpi veergudes olevad arvud põhjustavad arvutustes pisikesi vigu, mis hakkavad kuhjuma ja suure tehingute arvu korral võivad põhjustada kellelegi arvestatavat teenimatut majanduslikku kahju/kasu.
- Meiliaadress on kuni 254 märki.
 - <https://stackoverflow.com/questions/386294/what-is-the-maximum-length-of-a-valid-email-address>
- Kui Teil on tabelis veerud nagu *xxx_kood*, *nimetus*, *kirjeldus*, mis on kõik tekstitüüpi, siis pole loogiline et neil kõigil on ühesugune väljapikkus. Ilmselt on kood lühike, nimetus veidi pikem ja kirjeldus kõige pikem.
- *Täisarvu tüüpi* võtmeveergude tüübi määramisel tuleb mõelda võimalikule ridade arvule tabelis. Tüüp tuleks valida nii, et vähemalt lähema viie aasta perspektiivis poleks vaja seda tüüpi muuta. Erinevat tüüpi andmeid sisaldavates tabelites on erinev hulk andmeid. Järgnev on väga "jäme" hinnang.
 - Klassifikaatorid (nt keeled, riigid, ametikohad, kauba seisundi liigid): üksikud read kuni tuhanded read. Sobiv tüüp SMALLINT või NUMBER(1)–NUMBER(5)
 - Põhiandmed (nt isikud, kaubad, teenused, teavikud, sõiduvahendid): tuhanded kuni miljonid read. Sobiv tüüp INTEGER/SERIAL või NUMBER(5)–NUMBER(10)
 - Transaktsioonilised andmed (nt tellimused, teadmiste kontrollid, taotlemised, paiknemised) ja sündmuste logid (nt seisundimuudatuste ajalugu): kümned tuhanded kuni sajad miljonid read. Sobiv tüüp INTEGER/SERIAL või BIGINT/BIGSERIAL või NUMBER(n), kus $n \geq 10$.
- Tekstivälja puhul, mille pikkus on üks märk, on loogilisem CHAR(1) kui VARCHAR(1). Lisaks peab olema CHECK kitsendus (vt ülesanne 3), et veerus ei tohi olla tühimärkidest koosnev string. Tühi string asendatakse CHAR tüübi korral tühiku(te)ga.
- TIMESTAMP ja TIMESTAMP WITH TIME ZONE tüüpi veergude puhul saab määrata täpsuse, mis näitab salvestatavate sekundi murdosade arvu. PostgreSQL korral saab see olla kuni (ja on vaikimisi) 6 kohta ning Oracle puhul saab see olla kuni (ja on vaikimisi) 9 kohta. Kui vastavalt nõuetele pole vaja sekundi murdosasid salvestada, siis peaks täpsus olema 0 – veeru tüübi definitsioon on TIMESTAMP(0) või TIMESTAMP(0) WITH TIME ZONE. Lugege ka:
 - https://maurus.ttu.ee/vaata_kysimust.php?kysimus=2873

Väike andmetüüpide vastavustabel.

PostgreSQL	Oracle (12.1)
SMALLINT	NUMBER(5)
INTEGER	NUMBER(10)
DECIMAL(19,4) NUMERIC(19,4)	NUMBER(19,4)
VARCHAR(100)	VARCHAR2(100 CHAR)
CHAR(3)	CHAR(3)
TEXT	CLOB
DATE	DATE
TIME	DATE
TIMESTAMP	DATE TIMESTAMP
BOOLEAN	NUMBER(1) (lisaks CHECK kitsendus, mis lubab veerus ainult väärtuseid 0 ja 1) Tüüpi Boolean saab veeru tüübina kasutada alates Oracle 23ai.

- 🚩 Kui soovite hoida PostgreSQL andmebaasis geograafilisi koordinaate, siis kaaluge tüübi *Point* kasutamist:
 - https://maurus.ttu.ee/vaata_kysimust.php?kysimus=2917
- 🚩 Kui EA vahendis pole diagrammil näha väljapikkuseid, kuigi mudelis on need määratud, siis toimige selle video juhise järgi.
 - [EA \(ver 12\) vahendi andmebaasi disaini mudelis väljapikkuste nähtavaks tegemine.mp4](#)

3.9 Piltide andmebaasis hoidmine

- 🚩 Kui Teie andmebaasis on vaja hoida pilte, siis mõelge veelkord läbi kuidas seda teha (ning vajadusel parandage ka kontseptuaalset andmemudelit). Võimalused on hoida andmebaasis järgnevat.
 - *Pildifaili* (andmebaas andmemahu mõttes suurem, kuid pilte käsitletakse kui ülejäänud andmeid – saab kasutada andmebaasisüsteemi pakutavaid turvalisuse, transaktsioonide ning varundamise/taastamise mehhanisme).
 - PostgreSQL korral veeru andmetüüp BYTEA või OID; Oracle korral tüüp BLOB.
 - *Pildifaili aadressi* (andmebaas andmemahu mõttes väiksem, võibolla andmete lisamise ja kuvamise mõttes esmapilgul lihtsam realiseerida, kuid turvalisuse, transaktsioonide ning varundamise/taastamise teemad nõuavad eritähelepanu, sest pildid ei ole andmebaasisüsteemi hoole all ja seega nende teemadega tegelemiseks ei saa andmebaasisüsteemi võimalusi kasutada).

- Andmebaasis selleks tekstitüüpi veerg. Väljapikkuse valimisel tuleb arvestada failisüsteemi lubatava kataloogitee ja failinime maksimaalse pikkusega:
 - <https://blog.codinghorror.com/filesystem-paths-how-long-is-too-long/> või kui soovite hoida URLi, siis selle maksimaalse pikkusega
 - <https://stackoverflow.com/questions/417142/what-is-the-maximum-length-of-a-url-in-different-browsers>
- ⚠ Erinevates operatsioonisüsteemides võib maksimaalne lubatud kataloogitee ja failinime pikkus erineda. Sellest tulenevalt võiks veerg olla tüüpi TEXT (PostgreSQL) või CLOB (Oracle) koos CHECK kitsendusega, mis määrab maksimaalse lubatud pikkuse. See lubab vajadusel maksimaalset lubatud pikkust kiiremini ja mugavamalt muuta. Süsteem võib eluea jooksul kolida ühelt füüsiliselt serverilt teisele, serveris võib muutuda operatsioonisüsteem, uues operatsioonisüsteemi versioonis võivad olla lubatud teistsugused pikkused.
 - <https://superuser.com/questions/14883/what-is-the-longest-file-path-that-windows-can-handle>
 - <https://www.quora.com/What-is-the-longest-file-path-allowed-for-Linux>
- Bill Karwini antimustrite raamat käsitleb failide hoidmist väljaspool andmebaasi antimustrina. Huvi korral lugege peatükki 12 raamatust:
 - ⚠ <https://learning.oreilly.com/library/view/sql-antipatterns-volume/9798888650011/?ar>
 - ⚠ https://www.ester.ee/record=b2888667*est
- Selles bakalaureusetöös antakse eelnimetatud lahendustest struktureeritud ja mõõtmistel põhinev ülevaade. Lisaks failide hoidmisele kohaliku serveri failisüsteemis käsitletakse ka võimalust hoida viidet pilvesüsteemi salvestatud failile. Autor hindab seda lahendust isegi eelistatavaks, eeldusel, et usaldatakse oma faile välise teenusepakkuja hoolde.
 - <https://digikogu.taltech.ee/et/Item/409ee372-2a60-4b69-8cd7-d0683b887c84>

3.10 Veergude kohustuslikkus

- ⚠ Mõelge läbi, millised veerud on kohustuslikud (NOT NULL) ja millised mitte. Kõik sellised valikud peavad olema kooskõlas kontseptuaalse andmemudeliga. Vajadusel tuleb teha muudatusi kontseptuaalses andmemudelis. EA vaikimisi mudeliteisendus jätab tabelite kirjelduses kõik mitte-primaarvõtme veerud mittekohustuslikuks (lubavad NULLe). Andmebaas tuleks disainida nii, et enamik veerge (esimeses järjekorras sageli otsingutingimustes kasutatavad veerud) on kohustuslikud – seega paljudele veergudele tuleb kohustuslikkus käsitsi määrata.
- Mõelge läbi, kas algselt mittekohustuslikuks mõeldud veerus võiks kasutada vaikimisi mingit väärtust, mis lubab deklareerida selle veeru kohustuslikuks.

Näide:

Kui tabelis on veerud *lisamise_aeg* ja *viimase_muutmise_aeg*, siis võiks mõelda nii, et lisamine on ühtlasi viimane muutmine. Seega mõlemad veerud võiksid olla kohustuslikud ja mõlemal veerul võiks olla vaikimisi väärtus, mis leiab hetke ajatempli.

Näide:

PostgreSQLis kuuluvad DATE, TIMESTAMP ja INTERVAL tüüpidesse ning ujukomatüüpidesse spetsiaalväärtused *-infinity* ja *infinity* (negatiivne ja positiivne lõpmatus). *-infinity* on kõikidest teistest tüübi väärtustest väiksem ja *infinity* on kõigist teistest tüübi väärtustes suurem. Sellised väärtused võimaldavad vältida NULLide kasutamist ja tänu sellele on näiteks päringute kirjutamine lihtsam.

<https://www.postgresql.org/docs/current/datatype-datetime.html#DATATYPE-DATETIME-INPUT>

```
CREATE TABLE Broneering (ruum INTEGER NOT NULL,  
alguse_aeg TIMESTAMP NOT NULL,  
lopu_aeg TIMESTAMP NOT NULL DEFAULT 'infinity',  
CONSTRAINT pk_broneering PRIMARY KEY (ruum, alguse_aeg),  
CONSTRAINT chk_broneering_vahemik CHECK  
(lopu_aeg >= alguse_aeg));
```

Broneeringu lõpu aeg võib olla teadmata – antud juhul on siis lõpu aja väärtus suurem kui kõik teised lõpu ajad.

Registreerin broneeringu, mille lõpu aeg pole teada (saab tänu vaikimisi väärtuse mehhanismile väärtuseks lõpmatuse).

```
INSERT INTO Broneering (ruum, alguse_aeg)  
VALUES (1, LOCALTIMESTAMP(0));
```

Millised ruumid on homse kella 00:00 seisuga broneeritud?

```
SELECT *  
FROM Broneering  
WHERE 'tomorrow' BETWEEN alguse_aeg AND lõpu_aeg;
```

'yesterday', 'today' ja 'tomorrow' on *süntaktiline suhkur*, mis teisendatakse süsteemi poolt konkreetseteks DATE või TIMESTAMP väärtusteks.

Sisestatud broneering on päringu tulemus.

Mis juhtuks siis, kui lõpu aeg oleks NULL (tabelis ei oleks *lopu_aeg* veerul NOT NULL kitsendust ja vaikimisi väärtust)? Sellisel juhul peab päringus sellega arvestama ja päring läheb tänu sellele keerukamaks (või kui ei arvestata, siis ei anna päring õiget tulemust).

Peaks näiteks kirjutama:

```
SELECT *
FROM Broneering
WHERE 'tomorrow' BETWEEN alguse_aeg
AND coalesce(lopu_aeg, 'infinity');
```

või

```
SELECT *
FROM Broneering
WHERE 'tomorrow'
BETWEEN alguse_aeg AND lopu_aeg
OR ('tomorrow' >= alguse_aeg
AND lopu_aeg IS NULL);
```

coalesce funktsioon asendab NULLi väärtusega.

Oracles temporaalsete (ajaandmete) tüüpide puhul selliseid spetsiaalväärtuseid kasutada ei saa ja lõpmatuse tähistamiseks tuleks valida mõni olemasolev andmetüüpi kuuluv väärtus (näiteks suurim võimalik väärtus)

<https://stackoverflow.com/questions/8011914/how-to-represent-end-of-time-in-a-database>

3.11 Primaarvõtmed ja unikaalsuse kitsendused

🚩 Igas tabelis peab olema primaarvõti. Kõik alternatiivvõtmed tuleb üles leida ja jõustada UNIQUE+NOT NULL kitsendustega.

- Tüüpviga on, et UNIQUE kitsendus dubleerib PRIMARY KEY kitsendust. Primaarvõti tähendab unikaalsust.
- Kasutajanime *tõstutundetu* unikaalsuse realiseerimiseks saab kasutada funktsioonil põhinevat UNIQUE indeksit (PostgreSQL, Oracle) või *exclusion* e EXCLUDE kitsendust (PostgreSQL). EXCLUDE on PostgreSQL jaoks eriomane kitsenduse tüüp, mis on unikaalsuse kitsenduse üldistus (st "tavaline" unikaalsus on selle erijuhus). Tõstutundetu unikaalsuse realiseerimine toimub ülesandes 10. Meiliaadressi tõstutundetu unikaalsus tähendab, et näiteks kui ühel isikul on meiliaadress erki.eessaar@taltech.ee, siis teisel ei saa see olla Erki.Eessaar@taltech.ee
 - Sellist indeksit ega ka *exclusion* kitsendust ei saa EA ning RR vahendis kirjeldada.
 - Märkige mudelis meiliaadress lihtsalt unikaalseks.
- Kui kontseptuaalne andmemudel sõnastab kitsenduse, mille kohaselt peab atribuudi väärtuse unikaalsus kehtima ainult teatud tingimustele vastavate olemite korral (näiteks, et aktuaalses või mitteaktuaalses seisundis kaupade nimetused peavad olema unikaalsed), siis selle realiseerimiseks saab kasutada osalist UNIQUE indeksit (PostgreSQL, Oracle) või *exclusion* e EXCLUDE kitsendust (PostgreSQL). Sellist indeksit ega ka *exclusion* kitsendust ei saa EA

ning RR vahendis kirjeldada. Seega tuleb sellise indeksi loomise kood ülesandes 4 ise kirjutada.

- PostgreSQL: <https://www.postgresql.org/docs/current/indexes-partial.html>
- PostgreSQL: <https://www.postgresql.org/docs/current/sql-createtable.html#SQL-CREATETABLE-EXCLUDE>
- Oracle: <https://dba-presents.com/index.php/databases/oracle/41-filtered-index-equivalent-in-oracle>
- Kui unikaalsuse kitsenduse jõustamiseks piisab UNIQUE kitsenduse loomisest, siis eelistage seda kindlasti unikaalse indeksi (UNIQUE INDEX) loomisele. Mõned praktilised põhjused.
 - Liiga lihtne on teha koodis muudatust CREATE UNIQUE INDEX => CREATE INDEX. Nii võib juhtuda, et andmebaasi arendamise käigus kaob unikaalsuse piirang märkamatuks ära.
 - Indeks pole SQL standardis kirjeldatud ja seega on suurem võimalus, et selle loomise lauset saab ühes andmebaasisüsteemis kasutada, kuid teises mitte.
 - Unikaalsuse kitsenduse defineerimisel tekib unikaalne indeks automaatselt.

⚠ Tuleb läbi mõelda, millistes tabelites on primaarvõtmed surrogaatvõtmed, millistes sisulised võtmed. Näiteks, kui CASE vahend pakub tabelisse *Isik* välja primaarvõtme veeru **isik_id** INTEGER, siis on Teie võimuses näiteks määrata, et primaarvõti on hoopis kombinatsioon **isikukood** VARCHAR(50) ja **isikukoodi_riik** CHAR(3) ning *isik_id* veerg kustutada. See omakorda viib kaskaadselt vajaduseni muuta selliste tabelite struktuuri, kus olev välisvõti peab otseselt või kaudselt viitama tabelile *Isik*. Paraku EA CASE vahendis on selliseid muudatusi **väga** tülikas teha ja seega Te ei pea oma projektis seda muutust tegema. **Klassifikaatorite koodid on sisulise tähendusega võtmed – neid ei genereeri süsteem, vaid sisestab teatud rollis lõppkasutaja!**

- Kui kasutate EA, siis saate andmebaasi füüsilise disaini mudelis määrata, et PostgreSQL andmebaasis on surrogaatvõtme veeru korral "tüüp" SERIAL (see on tegelikult notatsioon, mis tingib andmebaasis INTEGER tüüpi veeru loomise, arvujada generaatori loomise ning veeru ja generaatori sidumise vaikimisi väärtuse mehhanismi kaudu).
 - Töövihiku projektis on tabeli *Isik* veerg *isik_id* tüüpi SERIAL.

⚠ Kas surrogaatvõtme väärtustena võiks täisarvude asemel kasutada universaalselt unikaalseid identifikaatoreid (UUID, GUID)?

- Jah, see on võimalus, kuid nagu alati on valikul nii positiivsed kui negatiivsed tagajärjed.
 - https://maurus.ttu.ee/vaata_kysimust.php?kysimus=2879

⚠ Millal kasutada primaarvõtme veeru nimes järelliidet **_id**? CASE vahend võib sellise nimega veeru lisada kõikidesse tabelitesse, kuid Teie võimuses on see veerg ümber nimetada või veerg kustutada. Selleks peaks primaarvõti rahuldama **kõiki** järgneva tingimusi.

- Primaarvõti on lihtvõti.
- Primaarvõti on surrogaatvõti, mitte sisulise tähendusega võti. Selle võtme väärtuseid hakkab genereerima süsteem. Võtmeväärtused moodustavad monotoonselt kasvava/kahaneva täisarvude jada.

Näide:

- **kaup_id** – tabeli *Kaup* primaarvõtme veerg, millesse hakkab väärtuseid genereerima süsteem.
 - 🔴 Miks ei sobi lihtsalt nimi ID? Vastuse annab peatükk 4 – "ID Required" SQL antimustrite raamatus.
 - https://www.ester.ee/record=b2888667*est
 - <https://learning.oreilly.com/library/view/sql-antipatterns-volume/9798888650011/?ar>
 - Siis on Teil endal ka hiljem lihtsam – arvujada generaator on vaja siduda vaid nende primaarvõtme veergudega, mis ei kattu välisvõtmega ja mille nimi lõpeb järelliitega **_id**.

🔴 Millal kasutada primaarvõtme veeru nimes järelliidet **_kood**? Primaarvõti peaks rahuldama **kõiki** järgnevaid tingimusi.

- Primaarvõti on lihtvõti.
- Primaarvõti on sisulise tähendusega võti. Sellesse veergu hakkab väärtuseid sisestama inimkasutaja.

Näide:

- **kauba_seisundi_liik_kood** – kauba seisundiklassifikaatori tabeli primaarvõtme veerg, millesse hakkab andmeid lisama piisavate õigustega inimkasutaja.
- Alternatiivne nimetamise strateegia on kasutada klassifikaatorite tabelite nimedes järelliidet **_R** (R nagu *reference data*) ning sellise tabeli primaarvõtme veeru nimes järelliidet **_r_id**.

🔴 Kui nimetate ümber võtmeveerge, siis tagage, et vastavalt nimetatakse ka ümber seotud välisvõtme veerud. Kui need nimed on ühesugused, siis on lihtsam kirjutada tabelite ühendamiseks mõeldud lauseid. Muudatus tuleb teha kõigepealt primaartabelis (välisvõtme poolt viidatavas). RR suudab selle nime muudatuse automaatselt üle kanda sõltuvasse (välisvõtit sisaldavasse) tabelisse (eeldusel, et mudelis on välisvõti korrektselt kirjeldatud). EAs tuleb see muudatus teha ka sõltuvas tabelis käsitsi.

Näide:

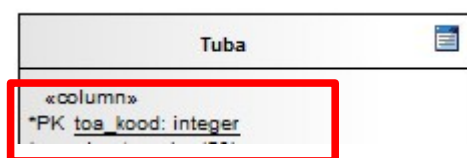
[Kauba_seisundi_liik]-1-----0..*-[Kaup]

Kõigepealt tuleb teha muudatus tabelis *Kauba_seisundi_liik* ning siis tabelis *Kaup*.

🔴 Võti võib olla liitvõti (näiteks isikukoodi ja selle väljastanud riigi koodi kombinatsioon tabelis *Isik*). Liitvõtme kohta võib vaadata videot:

- EA (12):
 - [030 Andmebaasi diagramm Liitvoti EA12.mp4](#)
- RR:
 - [270 Andmebaasi diagramm Liitvoti.mp4](#)

- ⚠ Ärge kirjeldage üksteist dubleerivaid võtmeid. Primaarvõti tähendab juba unikaalsust. Primaarvõtme veergudele lisaks UNIQUE kitsenduse deklareerimine oleks viga. Tulemuseks võivad olla üksteist dubleerivad indeksid. Järgnevalt on illustreeritud vea esinemist EA andmebaasi disaini mudelis. PK veeru nime ees näitab, et veerg kuulub primaarvõtmesse. Veeru allajoonimine näitab, et see veerg kuulub UNIQUE kitsendusse.



3.12 Välisvõtmed

- ⚠ Kõik välisvõtmed tuleb jõustada FOREIGN KEY kitsendustega.
- ⚠ Jälgige välisvõtme puhul, et selles osalevatel primaarse ja sõltuva tabeli veergudel oleksid ühesugused või ühilduvad andmetüübid. Andmetüüpide muutmisel muutke *primaartabeli* võtmeveeru tüüpi. RR ja EA suudavad selle muudatuse automaatselt sõltuva tabeli kirjeldusse üle kanda (eeldusel, et mudelis on välisvõti korrektselt kirjeldatud). Kui muudate välisvõtme veeru tüüpi sõltuvas tabelis, siis EA ei kanna seda muudatust automaatselt primaartabelisse üle. RR ei lase sellist muudatust teha.

Näide:

[Kauba_seisundi_liik]-1-----0..*-[Kaup]

Kui soovin määrata, et *kauba_seisundi_liik_kood* on tüüpi SMALLINT, mitte INTEGER, siis teen muudatuse tabeli *Kauba_seisundi_liik* kirjelduses.

Tähelepanu: Kui kasutate PostgreSQL'i ning:

- ⚠ primaartabelis on võtmeveeru tüüp SMALLSERIAL, siis peab sõltuvas tabelis vastav välisvõtme veerg olema tüüpi SMALLINT,
- ⚠ primaartabelis on võtmeveeru tüüp SERIAL, siis peab sõltuvas tabelis vastav välisvõtme veerg olema tüüpi INTEGER,
- ⚠ primaartabelis on võtmeveeru tüüp BIGSERIAL, siis peab sõltuvas tabelis vastav välisvõtme veerg olema tüüpi BIGINT.

Kui määrate EAs veeru "tüübiks" SERIAL / SMALLSERIAL / BIGSERIAL, siis määrab CASE vahend selle "tüübi" automaatselt ka sellele veerule viitavatele välisvõtme veergudele. Välisvõtme veergude tüübid tuleb käsitsi asendada vastavalt tüüpidega INTEGER / SMALLINT / BIGINT.

- ⚠ Välisvõtmete puhul tuleb määrata sobivad kompenseerivad tegevused.

- Kui **sama olemi andmed on laiali mitmes tabelis**, siis neid omavahel siduvate välisvõtmete korral võiks kasutada ON DELETE CASCADE.
 - Näiteks oletame, et kaubaartiklite kohta on andmed tabelites *Kaup*, *Kauba_kategooria_omamine* ja *Kauba_omadus*. Kui tahan kauba *k* andmed andmebaasist kustutada, siis pean kustutama *k* kohta käivad read kõigist nendest tabelitest. Oleks ju lihtsam, kui peaksin kustutama kauba andmed ainult tabelist *Kaup* ja süsteem hoolitseb automaatselt ülejäänud andmete kustutamise eest!
 - Kontseptuaalses andmemudelis olnud **üldistuste** alusel tekkinud välisvõtmetele ON DELETE CASCADE.
 - Kontseptuaalses andmemudelis **kompositsiooniseoste** alusel tekkinud välisvõtmetele ON DELETE CASCADE.
- Kui välisvõti **viitab sisulise tähendusega võtmele** (klassifikaatori kood, isikukood, auto registrikood, dokumendi number, üliõpilaskood, ...), siis ON UPDATE CASCADE.
 - seda saab määrata PostgreSQLis, kuid ei saa määrata Oracles.
- Kui välisvõtme poolt realiseeritava seosetüübi korral on püstitatud reegel, et **vanema andmete kustutamisel peavad selle laste andmed säilima**, siis on sobiv määrang ON DELETE SET NULL.
- Kõigil ülejäänud juhtudel sobib vaikimisi määrang ON UPDATE/DELETE NO ACTION.
 - Näiteks oletame, et *isik_id* väärtused tabelisse *Isik* genereerib süsteem. Esiteks, *isik_id* väärtuseid pole kellelgi põhjust muuta. Teiseks, kui *isik_id* väärtust ikkagi muuta, siis võib tekkida olukord, et süsteem genereerib uue rea lisamisel veeru jaoks väärtuse, mille keegi on sinna juba lisanud. Sellised read jäävad tabelisse lisamata. Kokkuvõtteks, antud juhul puhul pole andmete muutmise kompenseerivat tegevust vaja (st ON UPDATE NO ACTION).
 - Lugege RESTRICT/NO ACTION erinevuste kohta. Soovitan ilma vajaduseta neid andmebaasis läbisegi mitte kasutada, st kasutage kas ühte või teist – muidu tekitate asjatut segadust.

🔗 https://maurus.ttu.ee/vaata_kysimust.php?kysimus=2927

3.13 Hierarhiliste andmete andmebaasis hoidmine

- 🔗 Kui kontseptuaalses andmemudelis on olemitüübil rekursiivne üks mitmele seosetüüp iseendaga (näiteks igal töötajal on null või üks mentor, kes on samuti töötaja), siis see viitab vajadusele hoida andmebaasis hierarhilisi andmeid. Levinud (võibolla isegi kõige levinum) lahendus SQL-andmebaasides on panna tabelisse (nt *Tootaja*) välisvõti, mis viitab sama tabeli primaarvõtmele. Sellist disainilahendust nimetatakse külgnevusnimistuks (*adjacency list*). Kuigi sellel lahendusel on ka puuduseid (vt Karwini antimustrite raamatu <https://learning.oreilly.com/library/view/sql-antipatterns-volume/9798888650011/?ar> peatükki 3), kompenseerib seda lahenduse populaarsus ja seega võrgus leiduvate juhendite ning näidete rohkus. PostgreSQLis

ja Oracles saab kasutada rekursiivseid päringuid, mis kahtlemata muudab külgnevusnimistu disainiga andmebaasi palju lihtsamini kasutatavaks.

- <https://neon.com/postgresql/postgresql-tutorial/postgresql-recursive-query>
- <https://oracle-base.com/articles/misc/hierarchical-queries>

Huvi (pole kohustuslik!) korral lugege järgnevatest lõputöödest selle kohta, kuidas esitada hierarhilis andmeid SQL-andmebaasides. Krönström (2015) magistritöös esitatakse struktureeritult kümme disainilahendust hierarhiliste andmete esitamiseks SQL-andmebaasides. Muide, need lahendused ei sisalda võimalust hoida SQL-andmebaasis hierarhilisi andmeid JSON või XML tüüpi veerus, st võimalusi on veelgi rohkem. Hierarhiliste andmete esitamisest PostgreSQL andmebaasis JSON tüüpi veerus kirjutab oma lõputöös Ossipova (2016). Mõlemas lõputöös on lisaks disainilahenduste kirjeldustele ka eksperimendid nendega PostgreSQL (Krönström, Ossipova) ja Oracle (Krönström) andmebaasides.

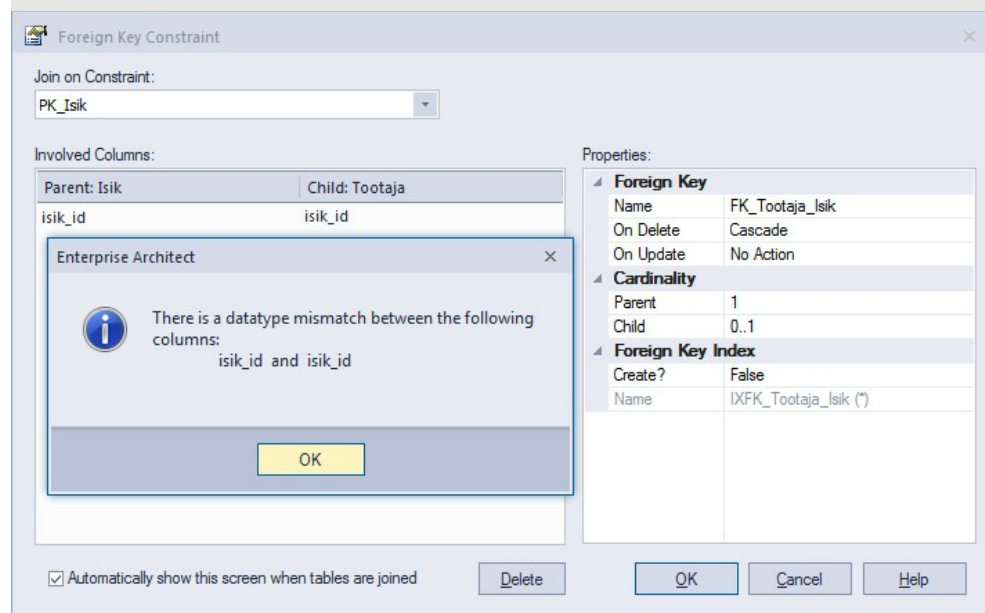
Krönström, K., 2015. *Hierarhiliste andmete esitamine SQL-andmebaasides kolme disainilahenduse näitel*. Magistritöö. TTÜ Informaatikainstituut.

[WWW] <https://digi.lib.ttu.ee/i/?3678> (29.08.2025)

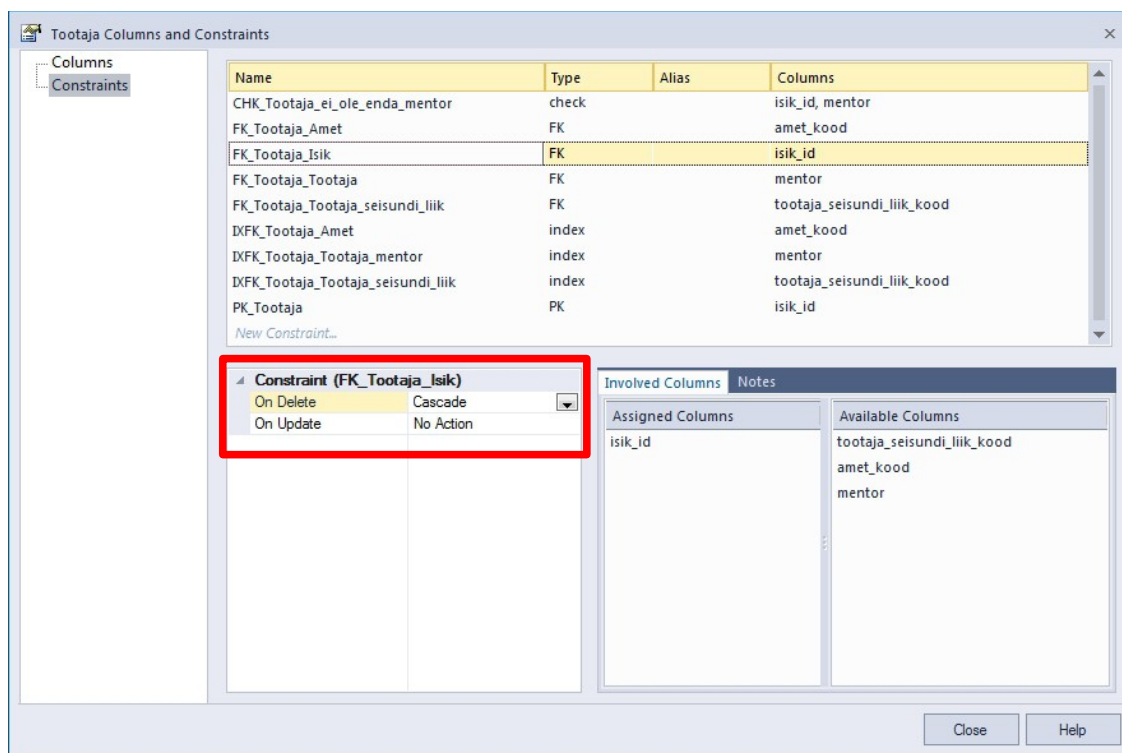
Ossipova, M., 2016. *Mitme väite ühe andmeväärtusena esitamise eelised ja puudused SQL-andmebaasides*. Bakalaureusetöö. TTÜ Informaatikainstituut.

[WWW] <https://digi.lib.ttu.ee/i/?6997> (29.08.2025)

Küsimus EA kasutamise kohta: "Nimelt, juhendi järgi peaks olema primaartabelis (nt *Isik*) surrogaatvõtme veerg (nt *isik_id*) SERIAL notatsiooniga ning tabelis (nt *Tootaja*), mille välisvõti viitab sellele veerule, peaks olema vastav välisvõtme veerg tüüpi INTEGER. Viisin muudatused sisse, aga kui hakkan lisama kompenseerivat tegevust sellele seosele ON DELETE CASCADE, ON UPDATE NO ACTION, siis EA ei lase =>ütleb, et *datatype mismatch*. Kuidas käituda?"



Vastus: Viga tekib, sest selles aknas muudatust tehes käivitub andmetüüpide sobivuse kontroll, mis ei saa aru, et antud juhul on kõik korras. Muutke kompenseerivat tegevust tabeli disainimise aknas, kitsenduste (*constraints*) alajaotuses. See ei käivita andmetüüpide kontrolli.



Vaadake pikemat selgitust videost:

[EA \(ver 12\) vahendis andmebaasi disaini mudelis välisvõtme kitsendustega seotud kompenseerivate tegevuste määramine.mp4](#)

3.14 Vaikimisi väärtused

Seal kus vaja, tuleb veerule määrata vaikimisi väärtus (nt välisvõtme veerule, mis viitab seisundiklassifikaatorile või veerule, kus registreeritakse iga rea korral selle loomise aeg). Samas, kui tahate näiteks registreerida, milline töötaja registreeris süsteemis millise olemi, siis välisvõtmel (registreerija_id) vaikimisi väärtust ei ole, sest välisvõtme väärtus tuleb määrata rakenduse poolt vastavalt sellele, kes sisse logis ja registreerimise tegevust läbi viis. RR vahendis loodud mudelis kirjutatakse vaikimisi väärtus *Default Value* välja ning EA vahendis loodud mudelis *Initial Value* välja.

- Kui veeru tüüp on `TIMESTAMP`, siis see on sama kui `TIMESTAMP WITHOUT TIME ZONE` ning sellisele veerule vaikimisi väärtuse leidmiseks on funktsioon `LOCALTIMESTAMP`, mitte `CURRENT_TIMESTAMP` või `Now()`.
- Kui Te ei soovi registreerida sekundi murdosi, siis kasutage `LOCALTIMESTAMP(0)` või `CURRENT_TIMESTAMP(0)`.
- Kui Te soovite vaikimisi väärtusena registreerida ajatempli minuti täpsusega, siis kasutage järgmisi avaldisi:
 - PostgreSQLis: `date_trunc('minute', LOCALTIMESTAMP)`
 - Oracles: `trunc(LOCALTIMESTAMP, 'mi')`
- Kui vaikimisi väärtus peaks olema hetke kuupäev, siis selle leiab funktsiooniga `CURRENT_DATE`.
- Transaktsioonidest e tehingutest.

- PostgreSQL funktsioonid `CURRENT_DATE`, `CURRENT_TIMESTAMP`, `LOCALTIMESTAMP` ja `Now()` tagastavad **transaktsiooni e teingu alguse aja**. See tähendab, et kui transaktsiooni käigus lisatakse tabelisse mitu rida (ühe või mitme lausega), siis saavad kõik need read sama ajatempli. PostgreSQLis on ka funktsioonid `statement_timestamp()` (uus ajatempel küsitakse iga lause alguses; kõigil selle lausega lisatud ridadel on sama ajatempel) ja `clock_timestamp()` (iga lisatava rea puhul küsitakse ajatemplit eraldi). Proovige järgnevat lauset ning näete, et veerus `current_timestamp` on samad väärtused ja veerus `clock_timestamp` on erinevad väärtused:

```
SELECT current_timestamp, clock_timestamp()
FROM generate_series(1, 10);
```

- Oracle funktsioonid `CURRENT_DATE`, `CURRENT_TIMESTAMP`, `LOCALTIMESTAMP` ja `SYSTIMESTAMP` tagastavad **lause alguse aja**. Kõigil selle lausega lisatud ridadel on sama ajatempel.
- Leian, et registreerimise aja määramine transaktsiooni alguse (PostgreSQL) või lause alguse (Oracle) täpsusega on sobiv ja isegi soovitatav. Kui ridu lisatakse ükshaaval, siis vahet ei ole. Kui korraga lisatakse mitu rida, siis aitab see hiljem kindlaks teha, millised read üheskoos lisati. Transaktsioon on loogiline tervik ja ma ei näe probleemi selles, et kõigil selle käigus lisatud ridadel on sama registreerimise aeg.
- Eelnevad näited määravad registreerimise aja serveri kohaliku aja järgi (nt kui server on Eestis, siis Eesti aja järgi). Kui soovite registreerimise aega maailmaaja järgi, siis ka seda on võimalik vaikimisi väärtusega määrata.
 - <https://maurus.ttu.ee/download.php?aine=346&document=36309&tyyp=do>
- Vajadusel muutke kontseptuaalse andmemudeli atribuutide definitsioonis atribuudi `reg_aeg` sõnalist definitsiooni ja näiteväärtust.
- Jälgige, et Te ei määraks staatilist vaikimisi väärtust veerule, mis moodustab primaarvõtme või millel on unikaalsuse kitsendus. Vaikimisi väärtuse mõte on, et see võib paljudes ridades korduda. Kuid eelnimetatud veergudes peab igas reas olema unikaalne väärtus.
 - Kui näiteks soovin määrata, et uued kaubad tuleb luua seisundis koodiga 10, siis määran vaikimisi väärtuse 10 tabeli `Kaup` veerule `kauba_seisundi_liik_kood`, mitte tabeli `Kauba_seisundi_liik` veerule `kauba_seisundi_liik_kood`.
- Ärge unustage, et tekstilised vaikimisi väärtused tuleb panna ülakomade e apostroofide vahele (nagu nt 'AKT') ning ülakomad tuleb määrata juba mudelis.
 - Jutumärkide kasutamine (nt "AKT") oleks viga, sest SQLis võivad standardi järgi jutumärkides olla identifikaatorid (nimed), kuid mitte tekstilised väärtused.
- Kümnenndmurd tuleb kirja panna kasutades punkti, mitte koma:
 - Vale: 2,2
 - Õige: 2.2
- Tõeväärtustüüpi veerule `on_nous_tylitamisega` sobib vaikimisi väärtus `FALSE`. 2018. aasta mais jõustus Euroopa Liidu tasemel andmekaitse üldmäärus, mis karmistab oluliselt isikuandmete töötlemiseks vajaliku nõusoleku tingimusi

(<https://arileht.delfi.ee/news/triniti/andmekaitsemaarus-muutub-tootlemiseks-vajaliku-andmete-omaniku-nousoleku-tingimused-karmistuvad?id=75871155>). "Määruse kohaselt kehtib nõusolek isikuandmete töötlemiseks ainult juhul kui selleks on antud selge avaldus, kinnitus (olgu siis suuliselt, elektrooniliselt või kirjalikult) või nõusolekut väljendav tegevus. Nõusolek ei saa tuleneda isiku vaikimisest või tegevusetusest. Seega on määruse mõtte kohaselt nõusolek antud nt veebisaidil kastis linnukese või risti tegemisega, kuid vabatahtliku nõusolekuga ei ole tegemist, kui linnuke või rist kastis on juba eelnevalt olemas (ja isik peaks keeldumiseks selle ise eemaldama)."

4 Lisainfo

Selle ülesande lahendamiseks saab lisainfot lisamaterjalide lehelt (kasutajanimi: SIS2, parool: SIS2):

- ✦ dokumendist **Andmebaasiobjektide nimetamine SQL-andmebaasides**
 - <https://maurus.ttu.ee/download.php?aine=346&document=31953&tyyp=do>
- ✦ slaidikomplektist **Baastabelid_süsteemikataloog**
 - <https://maurus.ttu.ee/download.php?aine=346&document=32472&tyyp=do>